Meta-Prompted Instruction Refinement: Bridging Manual Prompting Techniques and Automatic Prompt Optimization

Linh Nguyen[a], Quang-Vinh Dang[b,*], Minh Ngoc Dinh[c] and Thuy Nguyen[d]

[a]School of Science, Engineering & Technology, RMIT University Vietnam, 702 Nguyen Van Linh Boulevard, Ho Chi Minh City 700000, Vietnam,
[b]British University Vietnam, Hung Yen, Vietnam,
[c]Millenia Education, Ho Chi Minh City, Vietnam,

ARTICLE INFO

Keywords:
Prompt engineering
meta-prompting
automatic prompt optimization
large language models
Big-Bench Hard

ABSTRACT

Large language models (LLMs) are transforming artificial intelligence by enabling systems that can reason, write, and assist with complex tasks, capabilities that are increasingly important for science, education, and everyday applications. Yet these models remain critically dependent on the quality of their input prompts, making prompt design a central bottleneck. Manual prompt engineering, using techniques such as chain-of-thought reasoning and role assignment, can yield high performance but requires expert knowledge and does not scale. Automatic prompt optimization (APO) offers efficiency, but its outputs often lack the structured guidance that makes human-crafted prompts effective. This paper introduces Meta-Prompted Instruction Refinement (MPIR), a lightweight, model-agnostic framework that refines APO-generated prompts through a seven-criteria rubric, meta-prompted evaluation and refinement, and empirical validation—without any additional training, search infrastructure, or access to model internals. Extensive experiments on the Big-Bench Hard (BBH) benchmark show that MPIR outperforms its PromptWizard baseline on 16 of 23 tasks, with gains of up to 20 percentage points on individual tasks, and that the same refinement layer improves two further APO methods (Iterative APE and ProTeGi) as well as a different underlying LLM family; the pooled average gain is directionally positive but not conclusively significant by paired statistical tests, so this per-task and cross-framework consistency, rather than the pooled figure alone, is offered as the primary evidence. Together, these results indicate that a simple, interpretable, post-hoc heuristic layer can improve prompts already optimized by heavier automated methods, bridging human heuristics with automation at a fraction of the engineering cost of learned or evolutionary alternatives.

1. 1. Introduction

Organizations adopting LLMs increasingly face a practical version of a familiar software-engineering problem: a component that works acceptably in a demo does not necessarily work reliably in production, and the gap between the two is often traceable to how the component is instructed rather than to a limitation of the underlying model. Unlike traditional software, an LLM has no fixed interface contract to program against; the prompt is simultaneously the specification, the interface, and—because prompts are natural language—an artifact whose correctness cannot be checked by a compiler or a type system. This makes prompt quality a cross-cutting concern that touches accuracy, consistency, cost, and user trust simultaneously, and makes the tooling used to construct and validate prompts a legitimate object of study in its own right, alongside the models the prompts are written for.

Large language models (LLMs) have rapidly advanced natural language processing, achieving strong performance on tasks such as translation (Goyal, Li and Durrett, 2023), summarization (Qin, Chen, Feng, Wu, Zhang, Li, Li, Che and Yu, 2025), and question answering (Zhang, Liu and Zhang, 2023), and are increasingly central to applied domains ranging from law to healthcare and decision support (Colombo, Pires, Boudiaf, Culver, Melo, Corro, Martins, Esposito, Raposo, Morgado and Desa, 2024; Zhang, Tian, Yang, Chen, Li and Petzold, 2025). Their effectiveness, however, depends critically on the design of the input prompt: even small changes in phrasing can produce large differences in output accuracy, consistency, and reliability (Errica, Sanvito, Siracusano and Bifulco, 2025). Prompt design has therefore become a major bottleneck to using LLMs reliably at scale.

*Corresponding author
s4021009@rmit.edu.vn (L. Nguyen); vinh.dq4@buv.edu.vn (Q. Dang); Minh.dinh@maeducation.com (M.N. Dinh); thuy.nguyen4@rmit.edu.vn (T. Nguyen)
ORCID(s):

Two broad strategies dominate current approaches to prompt design: manual prompting and automatic prompt optimization (APO) (Liu, Yuan, Fu, Jiang, Hayashi and Neubig, 2023). Manual prompting relies on heuristics drawn from human intuition and experience, such as chain-of-thought reasoning (Wei, Wang, Schuurmans, Bosma, Ichter, Xia, Chi, Le and Zhou, 2022), role assignment (Kong, Zhao, Chen, Li, Qin, Sun, Zhou, Wang and Dong, 2024), and few-shot demonstration (Brown, Mann, Ryder, Subbiah, Kaplan, Dhariwal, Neelakantan, Shyam, Sastry, Askell, Agarwal, Herbert-Voss, Krueger, Henighan, Child, Ramesh, Ziegler, Wu, Winter, Amodei et al., 2020). These techniques are often effective and interpretable, but require domain expertise (Reynolds and McDonell, 2021; Zamfirescu-Pereira, Wong, Hartmann and Yang, 2023) and substantial trial-and-error (Mayilvaghanan, Nathan and Kumar, 2025), and are difficult to scale across the growing range of LLM applications. APO instead automates prompt generation and refinement through iterative search or model feedback (Pryzant, Iter, Li, Lee, Zhu and Zeng, 2023; Guo, Wang, Guo, Li, Song, Tan, Liu, Bian and Yang, 2024), improving efficiency and scalability but often forfeiting the structured guidance that makes human-crafted prompts effective. The trade-off between the effectiveness and interpretability of manual prompting and the efficiency and scalability of APO remains unresolved.

Recent work on meta-prompting suggests a path forward: meta-prompts guide an LLM in generating, critiquing, or refining prompts, effectively adding a higher-order layer of reasoning to prompt optimization (Yang, Li, Li, Chen, Gao, Zhang, Li and Feng, 2024b; Mayilvaghanan et al., 2025). Yet most existing approaches integrate heuristics only loosely, either as initial seeds or as informal guidance during optimization, without systematic evaluation or explicit accounting for LLMs' sensitivity to subtle phrasing differences (Errica et al., 2025). As a result, their effects are difficult to quantify and to generalize. What is missing is a principled framework that embeds human heuristics into APO in a structured, repeatable, and empirically validated way.

This paper addresses that gap with Meta-Prompted Instruction Refinement (MPIR), a framework that integrates manual prompting heuristics into APO through a structured three-stage cycle of evaluation, refinement, and validation. MPIR functions as a modular layer on top of an existing APO system: it translates heuristics into explicit evaluation criteria, applies meta-prompted revision, and empirically validates each candidate prompt, thereby embedding human-guided design principles into APO-generated prompts without additional human intervention. Unlike the increasingly elaborate evolutionary, reinforcement-learning, and multi-agent machinery pursued elsewhere in this space (Section 2.5), MPIR asks how much of that benefit is recoverable from a fixed, interpretable checklist and a small, fixed number of meta-prompting calls—deliberately inexpensive to add on top of whatever APO pipeline a practitioner already has.

Specifically, this study pursues four objectives:

- **To develop a structured rubric for prompt evaluation:** formalizing established manual prompting heuristics into a seven-criteria framework.
- **To evaluate whether meta-prompted refinement improves APO-generated prompts:** applying rubric-guided feedback to iteratively refine and empirically validate prompts.
- **To investigate the generalizability and modularity of heuristic-guided refinement:** testing MPIR as a layer across different APO frameworks and underlying LLMs.
- **To investigate how individual prompting heuristics contribute to performance:** quantifying each rubric criterion's contribution via targeted ablations.

The remainder of this paper is organized as follows. Section 2 reviews related work on prompt engineering and automatic prompt optimization. Section 3 introduces the MPIR framework. Section 4 describes the implementation. Section 5 reports results and analysis. Section 6 discusses threats to validity. Section 7 concludes the paper.

2. 2. Related Work

2.1. 2.1. Prompting as Human-AI Interaction

Prompting is the primary mechanism for steering LLM behavior, leveraging in-context learning to shape outputs without updating model parameters (Liu et al., 2023). It has been described as a form of natural language programming (Reynolds and McDonell, 2021), emphasizing scalability and efficiency. As a direct channel of user control, however, its effectiveness often depends on extensive trial and error, which has given rise to a wide range of heuristic techniques whose potential and limitations we review next.

This framing matters for how MPIR is positioned. Because prompting is a form of programming without a compiler, the feedback a prompt author receives is indirect: a change in wording is only validated by re-running the model and inspecting its output, and there is no static analysis that flags an ambiguous instruction before it is executed. Studies of non-expert prompt authors find that this absence of structural feedback is a primary source of difficulty, with users struggling to predict how small wording changes will affect model behavior and often overfitting to a handful of examples they happen to test (Zamfirescu-Pereira et al., 2023). Manual heuristics such as those reviewed in Section 2.2 function, in this framing, as an informal type system for prompts: they encode the kinds of structural properties—an assigned role, a stated reasoning procedure, a specified output format—that experienced prompt authors have learned to check for even without automated tooling to enforce them. MPIR's seven-criteria rubric (Section 3.3) can be read as an attempt to make this informal type system explicit and machine-checkable, applying it as an automated review step after a prompt has already been produced by another method, in the same way a linter is applied after code has already been written rather than only informing initial code style.

2.2. 2.2. Manual Prompting Heuristics

Manual prompt engineering improves LLM performance through deliberate prompt design without modifying model parameters. Zero-shot prompting relies solely on task instructions (Brown et al., 2020), while few-shot prompting introduces demonstrations that improve generalization and output consistency (Winata, Huang, Vadlamannati and Chandarana, 2023); example formatting and label consistency further shape performance, underscoring the importance of prompt structure in activating latent model capabilities (Min, Lyu, Holtzman, Artetxe, Lewis, Hajishirzi and Zettlemoyer, 2022).

More advanced heuristics target reasoning structure and contextual alignment. Chain-of-thought (CoT) prompting improves multistep reasoning by decomposing problems into intermediate steps (Wei et al., 2022; Kojima, Gu, Reid, Matsuo and Iwasawa, 2022), and coherent, relevant reasoning steps matter more than reasoning length alone (Wang and Zhou, 2024; Wang, Min, Deng, Shen, Wu, Zettlemoyer and Sun, 2023a). Role prompting introduces contextual identities, such as "Act as a math teacher," to guide reasoning behavior and domain alignment (Wang, Mi, Chen, Xue, Wang, Zhu, Wong and Xu, 2024; Kong et al., 2024), although poorly aligned personas can degrade reasoning quality (Kim, Yang and Jung, 2024). Step-back prompting further improves knowledge-intensive reasoning by encouraging the model to first abstract key principles before solving a task (Zheng, Mishra, Chen, Cheng, Chi, Le and Zhou, 2024; Coda-Forno, Binz, Wang and Schulz, 2024).

Prompt organization matters as well: because LLMs attend disproportionately to information placed near the beginning or end of a prompt (Yu, Jiang, Luo, Wu, Lin, Li, Yang, Huang and Qiu, 2025; Liu, Lin, Hewitt, Paranjape, Bevilacqua, Petroni and Liang, 2024), strategically ordering instructions and constraints can improve reasoning accuracy and instruction-following. Separating context from task instructions—for example with explicit delimiters or headers—reduces ambiguity about which part of a prompt the model should treat as background versus as an actionable directive, a distinction that becomes more important as prompts grow longer and combine multiple heuristics at once.

A further line of heuristics concerns how in-context examples themselves are constructed rather than simply whether they are present. Demonstrations that walk through intermediate reasoning steps, not just final answers, teach a model the expected reasoning process as well as the expected output format, and models trained or prompted to mimic worked examples tend to reproduce that structure on novel inputs (Winata et al., 2023; Min et al., 2022). Explicitly specifying the desired output format—for instance, requiring a delimiter-wrapped final answer—further reduces downstream parsing errors and makes automated grading more reliable, which matters directly for benchmark evaluation. Finally, closing a prompt with a brief restatement of the task exploits the same positional-attention effect that motivates placing key instructions early: a short concluding reminder keeps the task salient immediately before the model begins generating its answer, complementing rather than duplicating an opening statement of intent.

A separate family of heuristics operates at decoding time rather than at the level of prompt text. Self-consistency samples multiple independent reasoning paths for the same chain-of-thought prompt and selects the answer that appears most frequently across samples, rather than relying on a single greedy decode (Wang, Wei, Schuurmans, Le, Chi, Narang, Chowdhery and Zhou, 2023b). This is complementary to, rather than competing with, the prompt-level heuristics reviewed above: an ensembling strategy over samples cannot compensate for a prompt that systematically misdirects the model's reasoning, but it can reduce the variance of a well-designed prompt's output. MPIR does not incorporate sampling-based ensembling, using a single greedy decode at temperature 0 throughout (Section 4.5); combining rubric-guided prompt refinement with self-consistency decoding is a natural extension we did not evaluate and note as future work.

Collectively, these heuristics show that carefully designed instructions, reasoning structure, contextual framing, and exemplars can substantially improve LLM performance—but they remain labor-intensive, task-specific, and dependent on human expertise, which motivates automatic prompt optimization. Section 3.3 formalizes seven of the heuristics introduced in this subsection—role framing, step-back reasoning, guided chain-of-thought, instruction separation, output format specification, worked reasoning in examples, and task-closing restatement—into the explicit rubric MPIR uses to evaluate and refine prompts.

2.3. 2.3. Automatic Prompt Optimization

APO methods scale prompt refinement through iterative generation, evaluation, and selection (Ramnath, Zhou, Guan, Mishra, Qi, Shen, Wang, Woo, Jeoung, Wang, Wang, Ding, Lu, Xu, Zhou, Srinivasan, Yan, Chen, Ding, Xu and Cheong, 2025). Continuous approaches optimize embeddings but require access to model internals, whereas discrete approaches edit natural language directly and produce interpretable outputs (Cui, Zhang, Li, Sun, Lopez, Das, Malin and Kumar, 2025). Automatic Prompt Engineer (APE) uses an LLM to generate and refine instructions from input-output demonstrations (Zhou, Muresanu, Han, Paster, Pitis, Chan and Ba, 2023); ProTeGi mimics gradient descent with textual feedback to guide revisions (Pryzant et al., 2023); EvoPrompt adapts evolutionary algorithms to mutate and recombine prompts (Guo et al., 2024); and OPRO uses an LLM as a natural-language optimizer that iteratively generates and evaluates candidates conditioned on prior scores (Yang, Wang, Lu, Liu, Le, Zhou and Chen, 2024a). PromptWizard is a self-evolving, self-adapting framework that optimizes prompts through a feedback-driven critique and synthesis process (Agarwal, Magazine, Singh, Dani, Ganu and Nambi, 2025).

These methods differ substantially in how they generate candidates and how they decide which candidate survives. APE treats prompt generation as program synthesis over a small set of demonstrations, scoring candidates by how well they reproduce the demonstrated input-output behavior (Zhou et al., 2023). ProTeGi instead treats natural-language critique as a proxy for a gradient: it asks an LLM to diagnose why a prompt fails on a batch of examples, generates an edit in the direction the critique suggests, and performs a beam search over the resulting candidates (Pryzant et al., 2023). EvoPrompt and OPRO both frame optimization as a black-box search over the space of prompt strings—EvoPrompt borrows crossover and mutation operators from genetic algorithms (Guo et al., 2024), while OPRO instead conditions the LLM on a running history of previously tried prompts and their scores, asking it to propose an improvement in the style of an optimizer reading its own trajectory (Yang et al., 2024a). PromptWizard, the primary baseline used in this paper, combines several of these ideas in sequence: it iteratively refines an instruction using critique-based feedback similar to ProTeGi's, selects a diverse set of in-context examples, sequentially re-optimizes instruction and examples together, and finally generates and validates self-produced reasoning chains for the selected examples (Agarwal et al., 2025), described in full in Section 4.2.1.

Related frameworks pursue complementary strategies. DSPy treats LM pipelines as text-transformation graphs, compiling declarative modules into effective prompting, fine-tuning, and reasoning strategies (Khattab, Singhvi, Maheshwari, Zhang, Santhanam, Vardhamanan, Haq, Sharma, Joshi, Moazam, Miller, Zaharia and Potts, 2024). TextGrad performs automatic optimization by propagating natural-language feedback through a system, analogous to how PyTorch propagates gradients (Yuksekgonul, Bianchi, Boen, Liu, Lu, Huang, Guestrin and Zou, 2025). Self-Refine has an LLM iteratively critique and revise its own output, echoing how humans improve writing through successive drafts (Madaan, Tandon, Gupta, Hallinan, Gao, Wiegreffe, Alon, Dziri, Prabhumoye, Yang, Gupta, Majumder, Hermann, Welleck, Yazdanbakhsh and Clark, 2023). Reflexion lets an agent verbally reflect on its mistakes and store these reflections in memory to improve future decisions, rather than updating model weights (Shinn, Cassano, Gopinath, Narasimhan and Yao, 2023). Because these frameworks treat optimization largely as black-box search, their outputs can be less robust and transparent, and by prioritizing automation they risk neglecting the structured heuristics that make manual prompting effective.

2.4. 2.4. Meta-Prompting and Heuristic-Guided Automatic Prompt Optimization

Meta-prompting guides an LLM to generate or refine prompts using performance feedback (Cui et al., 2025), and several studies combine manual heuristics with automatic optimization. PE2 formalizes refinement through step-by-step reasoning templates and structured analysis (Ye, Ahmed, Pryzant and Khani, 2024); dual-phase strategies combine a meta-instruction-guided initialization phase with iterative sentence-level optimization to accelerate convergence on high-quality prompts (Yang et al., 2024b); and PROPEL integrates a large set of expert-derived prompting principles as priors to guide refinement (Mayilvaghanan et al., 2025). PromptWizard itself adopts heuristic-driven strategies to initialize prompts and self-generated reasoning chains to construct few-shot examples (Agarwal et al., 2025).

PE2 illustrates the mechanics of this category concretely: rather than asking an LLM to simply "improve this prompt," it supplies a meta-prompt containing an explicit two-step reasoning template—first diagnose specific problems with the current prompt using a structured checklist of failure modes, then propose a textual edit that addresses the diagnosed problems—which the authors show outperforms unstructured revision requests of similar length (Ye et al., 2024). PROPEL takes a different route to the same broad goal: rather than diagnosing a specific prompt's weaknesses at refinement time, it curates a set of expert-derived prompting principles in advance and supplies them as priors during a single optimization pass, so that the principles shape the search from the outset rather than critiquing an already-produced candidate (Mayilvaghanan et al., 2025). Both approaches, along with PromptWizard's own heuristic-driven initialization, treat manual prompting knowledge as an input to the search process itself, which is what distinguishes them from the earlier black-box APO methods in Section 2.3 but also what limits how cleanly their heuristic contribution can be isolated from their search contribution.

Important gaps remain, however. When heuristics are used at all, they are typically injected at initialization (Yang et al., 2024b) or loosely during optimization (Mayilvaghanan et al., 2025) rather than treated as an independent stage, so their contribution is difficult to attribute to measurable gains. Current approaches also rarely address LLMs' sensitivity to subtle phrasing differences (Errica et al., 2025), so reported improvements may not consistently transfer to task outcomes. Meta-prompting thus expands automation but still lacks a structured integration of heuristic knowledge. MPIR differs from its two closest relatives in this respect. PE2 is itself a complete, self-contained optimizer that formalizes one evaluation-refinement loop with its own held-out scoring, much like MPIR's own validation stage; the key difference is architectural rather than in whether validation is used at all—PE2 optimizes a prompt from scratch as a single end-to-end pipeline, whereas MPIR is explicitly designed to layer onto a prompt that a separate, already-completed APO run has already produced, making it agnostic to which upstream method generated that prompt. PROPEL instead bakes a large set of expert-derived principles into a single refinement pass as implicit priors, rather than scoring a prompt against each principle explicitly and iterating multiple evaluation-refinement-validation cycles until an empirically best candidate is found. MPIR's contribution is precisely this combination—an explicit, criterion-by-criterion rubric, applied as a separate stage on top of an existing APO output, with iterative empirical validation deciding which candidate survives—rather than any single component in isolation.

A closely related recent approach applies the same two-stage pattern—run a search-based optimizer, then locally refine its output—to few-shot relation extraction, using gradient- or attribution-style editing for the second stage rather than a fixed heuristic rubric scored by an LLM judge, and without testing generality across multiple upstream APO methods or backbone models (Chakma, Surdeanu and Blanco, 2026). MPIR shares its premise (a first-stage optimizer leaves room for post-hoc improvement) but differs in mechanism (a human-authored, criterion-by-criterion rubric evaluated by meta-prompting rather than local gradient-style edits) and in scope (evaluated across three upstream APO methods and two model families on a general reasoning benchmark rather than one task and one optimizer).

2.5. 2.5. The Shifting Frontier of Prompt Optimization

Since the meta-prompting and heuristic-guided methods above were introduced, prompt optimization has fragmented into more elaborate directions: evolutionary search with natural-language reflection, shown to outperform reinforcement-learning-based optimization with far fewer rollouts (Agrawal, Tan, Soylu, Ziems, Khare, Opsahl-Ong, Singhvi, Shandilya, Ryan, Jiang, Potts, Sen, Dimakis, Stoica, Klein, Zaharia and Khattab, 2026); reinforcement learning directly over edit actions (Lin, Lubis, Ruppik, van Niekerk, Shen, Heck, Vukovic, Feng and Gasic, 2025); multi-agent debate and tournament-style Elo ratings as richer fitness functions than single-judge scoring (Nair, Banerjee, Mombaerts, Hagen and Borogovac, 2025); explicit error taxonomies that guide refinement top-down rather than through a fixed checklist (Singh, Yadav and Blanco, 2026); and prompt format, not just content, as its own optimization axis (Liu, Xu, Zhang, Chen, Feng, Chen, Guo, Yang and Cheng, 2025). A recent survey frames the area through an optimization-theoretic lens (Li, Wang, Li and Jin, 2025), and there is growing interest in learned or instance-adaptive rubrics that replace fixed, human-authored criteria such as MPIR's seven. Relative to this frontier, MPIR's rubric is not the most sophisticated available mechanism; its contribution is instead that a small, interpretable, model-agnostic, and inexpensive post-hoc layer captures a meaningful share of the benefit these heavier methods pursue, without their training, search infrastructure, or per-task tuning cost—a different point on the cost-versus-sophistication trade-off, not a claim to surpass it.

Table 1 summarizes where MPIR sits relative to the methods discussed above, along five dimensions that recur throughout this section: whether the method is guided by an explicit, human-authored rubric rather than a learned or implicit one; whether it is agnostic to the specific APO method or model it is paired with, rather than being a

Table 1
Positioning of MPIR relative to representative prompt-optimization and refinement methods.

Method	Explicit rubric	APO-agnostic layer	Training-free	Held-out validation	Multi-round
APE ‹19›	No	No	Yes	Yes	Yes
ProTeGi ‹20›	No	No	Yes	Yes	Yes
EvoPrompt ‹21›	No	No	Yes	Yes	Yes
OPRO ‹22›	No	No	Yes	Yes	Yes
PromptWizard ‹23›	Partial	No	Yes	Yes	Yes
PE2 ‹24›	Partial	No	Yes	Yes	Yes
PROPEL ‹26›	Yes	No	Yes	No	No
ETGPO ‹60›	No	No	Yes	Partial	Yes
CFPO ‹61›	No	No	Yes	Yes	Yes
GEPA ‹59›	No	No	Yes	Yes	Yes
DEEVO ‹62›	No	No	Yes	Partial	Yes
MPIR (this work)	Yes	Yes	Yes	Yes	Yes

self-contained optimizer; whether it requires no additional training or fine-tuning; whether it retains empirical, held-out validation as part of candidate selection rather than relying on the judge's score alone; and whether it supports multiple iterative rounds rather than a single pass.

No prior method combines all five properties. PROPEL is rubric-guided but bakes its principles into a single pass without a separate validation stage; ETGPO organizes refinement around failure categories rather than a fixed rubric, and validates only within its own search loop rather than against a truly held-out set; and the remaining methods are self-contained optimizers rather than layers designed to sit on top of an arbitrary upstream APO output. MPIR's position in this table is also its main limitation: an explicit, model-agnostic, iteratively validated rubric is a simpler mechanism than reflective evolution or reinforcement learning over edits, and Section 5 shows this simplicity comes with a correspondingly modest effect size.

The literature reveals a persistent tension: manual prompting offers interpretability and effectiveness but does not scale, while automatic optimization offers efficiency and scalability but often neglects heuristic grounding. Emerging meta-prompting methods attempt to bridge this divide but lack systematic integration and principled evaluation. MPIR is designed to close this gap by formalizing manual heuristics into explicit rubrics, applying them as an independent stage of the optimization pipeline after an APO prompt has already been produced, and validating the resulting prompt's effectiveness on held-out data—combining human insight with automated scalability.

3. 3. Meta-Prompted Instruction Refinement Framework

We propose Meta-Prompted Instruction Refinement (MPIR), a framework that extends APO by embedding manual prompting heuristics into a structured, iterative refinement process (Figure 1). MPIR uses meta-prompting to guide an LLM in systematically evaluating, refining, and validating APO-generated prompts, and consists of three components: (1) a heuristic rubric encoding principles such as role prompting and chain-of-thought reasoning; (2) an evaluation-refinement loop in which meta-prompts critique and revise candidate prompts; and (3) a validation stage that measures refined-prompt effectiveness on a held-out set using accuracy.

These three stages echo concepts that appear under different names elsewhere in the prompt-optimization literature: evaluation relates to the critique-based, feedback-driven assessment used in frameworks such as ProTeGi and PromptWizard (Pryzant et al., 2023; Agarwal et al., 2025); refinement aligns with the revision and self-improvement strategies common to meta-prompting (Agarwal et al., 2025; Ye et al., 2024); and validation relates to the empirical, performance-based candidate selection used across APO frameworks (Zhou et al., 2023; Yang et al., 2024a; Agarwal et al., 2025). By organizing these components into one rubric-driven cycle, MPIR provides a more structured integration of heuristic-guided prompt refinement than prior work.

3.1. 3.1. Problem Formulation

Let (Qv, Av) = {(qi, ai)}, i = 1..M, denote a validation set of M question-answer pairs, where qi is the i-th input question and ai its ground-truth answer. An LLM L generates outputs with probability pL(ai | qi, P), where P is the

prompt under evaluation, and its performance is measured by a task-specific metric A(L, P) (accuracy) computed on the validation set. The goal of MPIR is to refine the APO-generated prompt P0 into a prompt Pfinal that maximizes this metric:

Pfinal = argmax over P' in R(P0) of A(L, P'),

where R(P0) denotes the space of candidate refinements derived from P0.

3.2. 3.2. MPIR Algorithm

Algorithm 1 details the iterative process of evaluation, refinement, and validation that produces the final refined prompt.

Algorithm 1 MPIR Algorithm

Require: L: target LLM; L': LLM for evaluation and refinement; Meval: meta-prompt for rubric evaluation; Mrefine: meta-prompt for refinement; P0: APO-generated prompt; (Qv, Av) = {(qi, ai)}, i=1..M: validation set; R: seven-criteria rubric; N: refinement rounds

Ensure: Refined prompt Pfinal

```
Initialize: Pfinal ← P0; Abest ← −∞
for t = 1 to N do
    S ← Evaluate(Meval, L', P0, R)          ▷ Rubric feedback
    P' ← Refine(Mrefine, L', P0, S)         ▷ Generate revised prompt
    A' ← Validate(L, P', (Qv, Av))          ▷ Performance of P'
    if A' ≥ Abest then
        Pfinal ← P'; Abest ← A'
    end if
end for
return Pfinal
```

Notation: P0 is the initial APO prompt used as the base for every refinement round; Pfinal is the best-performing prompt found across all rounds; P' is the candidate produced in the current round; S is the textual rubric feedback returned by Evaluate(); Abest is the best validation score observed so far (initialized to $-\infty$); and A' is the validation score of the current candidate P'.

3.3. 3.3. Seven-Criteria Evaluation Rubric

The rubric combines insights from academic research, industry practice, and iterative experimentation. Research provides a diverse catalog of prompting strategies (Schulhoff, Ilie, Balepur, Kahadze, Liu, Si, Li, Gupta, Han, Schulhoff, Dulepet, Vidyadhara, Ki, Agrawal, Pham, Kroiz, Li, Tao, Resnik et al., 2024; Sahoo, Singh, Saha, Jain, Mondal and Chadha, 2024), while industry contributes large-scale validation from deployed systems and user bases (Boonstra, 2025; Anthropic, 2025; OpenAI, 2025a; Microsoft, 2025); iterative testing then refined these insights into a form that generalizes across tasks. The resulting seven criteria are intentionally ordered as a coherent progression—from role framing and conceptual grounding, through structured reasoning and exemplification, to task closure—turning heuristics into a systematic framework for improving prompt quality:

- **Role Prompting** Assign a task-relevant role to condition model behavior.
- **Step-Back Reasoning** Encourage abstraction and articulation of core concepts.
- **Guided Chain of Thought** Structure reasoning into clear, sequential steps.
- **Instruction and Separation** Separate task instructions from context to reduce ambiguity.
- **Output Format with Examples** Specify the response schema and provide illustrative examples.
- **Worked Reasoning in Examples** Include step-by-step reasoning within exemplars.
- **Conclusion** Restate the task at the end to reinforce intent and coherence.

Each criterion is grounded in an established prompting technique: Role Prompting in contextual role assignment (Wang et al., 2024; Kong et al., 2024); Step-Back Reasoning in evidence that abstracting key principles improves reasoning (Zheng et al., 2024; Coda-Forno et al., 2024); Guided Chain-of-Thought in structured intermediate reasoning for complex tasks (Wei et al., 2022; Kojima et al., 2022); and Output Format with Examples and Worked Reasoning in Examples in few-shot and demonstration-based learning (Winata et al., 2023; Min et al., 2022). The Conclusion criterion follows from evidence of positional bias in LLMs, whereby information near the start or end of a prompt receives more attention (Yu et al., 2025; Liu et al., 2024). Consolidating these techniques operationalizes established manual heuristics into a systematic rubric that the next section applies to APO-generated prompts.

3.4. 3.3.1. Illustrative Application of the Rubric

To make the abstract rubric concrete before Section 3.4 formalizes how it is applied automatically, this subsection walks through how the seven criteria would assess the real PromptWizard-optimized prompt for penguins_in_a_ table reproduced in full in Appendix A.3, without reporting fabricated numeric scores for an evaluation that was not separately logged for this specific illustration. The PromptWizard prompt opens with a single undifferentiated paragraph that mixes a task description with an aside about comparing a new penguin's height to existing entries—an instruction that turns out to be irrelevant to most of the actual questions asked. Scored against the rubric: Role Prompting is weak, since no task-relevant persona is assigned; Instruction and Separation is weak, since context and directive are fused into one paragraph with no delimiter; Guided Chain of Thought is present only implicitly, in the numbered reasoning steps of the worked examples rather than in the instructions themselves; and Conclusion is absent, since the prompt ends abruptly after the worked examples with no restatement of the task. The MPIR-refined version of the same prompt, reproduced in Appendix A.4, directly addresses the two weakest criteria identified above: it opens with an explicit role ("You are a data analyst tasked with comparing the attributes of penguins..."), separates context from instructions under labeled headers (### Context, ### Instructions), and closes with an explicit ### Conclusion section restating the task. This is precisely the pattern reported quantitatively in Section 5.2.4 and Table 7: removing Instruction and Separation (C4) or Role Prompting (C1) causes the largest ablation drops, consistent with these being the criteria most visibly deficient in the unrefined baseline for this task.

3.5. 3.4. Evaluation and Refinement

The evaluation-refinement stage extends manual prompting strategies into a systematic process for improving APO-generated prompts, mirroring how a human prompt engineer would diagnose a prompt's weaknesses and revise it using proven heuristics.

The evaluation stage uses a meta-prompt (Figure 2) that directs the LLM to apply the seven-criteria rubric to an APO-generated prompt, scoring each criterion and identifying strengths, weaknesses, and actionable improvements. This produces a structured critique that combines quantitative scoring with qualitative insight, closely approximating expert human review, and reveals where the prompt departs from established heuristics.

The refinement stage builds on this critique. A second meta-prompt (Figure 3) directs the LLM to revise the original prompt by systematically incorporating the evaluation's suggestions, with the evaluation itself preserved in the conversation history to keep the revision faithful to the rubric rather than an unconstrained rewrite.

Evaluation and refinement together form a feedback-driven process for improving prompts, but refinement alone does not guarantee performance gains; MPIR therefore adds a validation stage to confirm that improvements translate into measurable benefits.

3.6. 3.5. Prompt Effectiveness Validation

The validation stage ensures that rubric-guided refinements yield measurable performance gains. Because LLMs are sensitive to subtle phrasing variations, a rubric-aligned rewrite is not guaranteed to perform better; rather than treating this sensitivity as a limitation, MPIR treats it as a mechanism for iterative improvement by testing each refined candidate on a held-out subset of the target dataset and comparing model outputs against ground truth. This evaluate-refine-validate cycle repeats for N rounds, producing successive candidates with corresponding accuracy scores, and MPIR retains the prompt that achieves the highest accuracy—so the final prompt reflects both heuristic soundness and empirically verified effectiveness.

4. 4. Implementation

4.1. 4.1. Datasets

This paper uses Big-Bench Hard (BBH) as the evaluation benchmark (Suzgun, Scales, Schärli, Gehrmann, Tay, Chung, Chowdhery, Le, Chi, Zhou and Wei, 2023). BBH comprises 23 tasks (27 subtasks) spanning algorithmic and multistep arithmetic reasoning, natural language understanding, world knowledge, and multilingual reasoning, with about 6,511 examples in total (roughly 250 per task). Its tasks are instruction-following in nature and were selected specifically for their difficulty, making BBH well suited for measuring gains from prompt optimization.

4.2. 4.2. Workflow

The workflow has two stages (Figures 4 and 5). Stage 1 uses PromptWizard as the base APO to produce an initial optimized prompt together with few-shot examples and reasoning chains. Stage 2 applies MPIR to that optimized prompt, running N rounds of rubric-based evaluation, refinement, and validation to produce the final optimized prompt.

4.2.1. 4.2.1. Stage 1: PromptWizard

PromptWizard is adopted as the base APO because it achieves strong performance across diverse benchmarks while remaining cost- and time-efficient (Agarwal et al., 2025). It is primarily an APO framework —its core objective is iterative, automated prompt refinement and validation—but it also incorporates meta-prompting and heuristic-guided elements through self-generated reasoning chains, critique-based refinement, and heuristic-inspired prompt construction. Its modular design lets MPIR be layered on top without altering the underlying optimization process. Our implementation omits the synthesis components of the original framework (Agarwal et al., 2025), since BBH's difficulty prevents the LLM from reliably generating them (for example, it often fails to produce a coherent expert persona). Algorithm 2 details the resulting procedure.

Concretely, RefineInstructions runs mutate_refine_rounds sequential rounds in which the current best instruction is mutated into style_variation stylistic variants and re-scored on a batch of training examples, keeping the best-performing variant for the next round; DiverseExampleSelection then samples a pool of candidate few-shot examples and greedily selects a subset that maximizes coverage of distinct failure modes rather than simply the highest-scoring examples; SequentialOptimization alternates between refining the instruction and re-selecting examples for max_seq_iter rounds, since the two interact—a better instruction can make a previously low-value example newly informative, and vice versa; and ReasoningComponent prompts the model to generate a step-by-step justification for each selected example's ground-truth answer, which ValidateComponent then filters to keep only examples where the generated reasoning actually arrives at the correct answer, discarding examples whose self-generated reasoning is unreliable even when the final answer label is correct by chance. The full hyperparameter values used for every stage of this procedure are listed in Table 2.

Algorithm 2 PromptWizard Algorithm (adapted from ‹23›)

Require: L: large language model; D: problem description; S = {(qi, ai)}, i=1..n: training samples; T: thinking styles; k: few-shot count; N1: max_seq_iter; N2: mutate_refine_rounds; Pbase: base instruction (zero-shot CoT)

Ensure: Optimized prompt $\hat{P}$opt and validated few-shot examples {(qfi, afi)}, i=1..k

1: Initialize: P ← Pbase
2: $\hat{P}$ ← RefineInstructions(L, D, S, T, N2)
3: Ediverse ← DiverseExampleSelection(L, D, S, $\hat{P}$)
4: $\hat{P}$opt, Eopt ← SequentialOptimization(L, $\hat{P}$, Ediverse, N1)
5: Eopt,r ← ReasoningComponent(Eopt) ▷ Generate reasoning chains
6: {(qfi, afi)} ← ValidateComponent(Eopt,r) ▷ Validate examples
7: **return** $\hat{P}$opt, {(qfi, afi)}, i=1..k

4.2.2. 4.2.2. Stage 2: Meta-Prompted Instruction Refinement

MPIR extends PromptWizard by applying the rubric-driven evaluation-refinement-validation loop (Algorithm 1) to its optimized prompt. In each round, the PromptWizard-generated prompt is evaluated against the seven-criteria rubric, refined using the meta-prompt feedback, and validated on a held-out set using accuracy; after N rounds, the best-performing prompt is selected.

4.3. 4.3. Baselines

Because the goal is to assess whether meta-prompting can enhance existing APO methods, PromptWizard is the primary baseline; additional reference points provide further context.

4.3.1. 4.3.1. Main baseline

- **APO baseline (PromptWizard).** PromptWizard serves as a strong automated baseline without human-crafted heuristics; each final prompt includes three automatically generated in-context examples.
- **Free rewrite baseline.** To isolate the contribution of MPIR's rubric from the general rewriting capability of GPT-4o, GPT-4o freely rewrites PromptWizard-generated prompts without rubric-guided evaluation or refinement, and the rewritten prompts are evaluated under the same BBH conditions as MPIR.

4.3.2. 4.3.2. Extended APO baselines

To further evaluate generalization, MPIR is also applied to APE (Zhou et al., 2023) and ProTeGi (Pryzant et al., 2023). Since neither method generates few-shot reasoning examples the way PromptWizard does, both use the same three chain-of-thought exemplars written by the BBH benchmark authors (Cobbe, Kosaraju, Bavarian, Chen, Jun, Kaiser, Plappert, Tworek, Hilton, Nakano, Hesse and Schulman, 2021); the resulting prompts are refined with MPIR and evaluated under identical conditions.

4.3.3. 4.3.3. Reference points

- **Manual prompting (zero-shot CoT).** A manual reference point using only the task description plus "Let's think step by step" (Kojima et al., 2022).
- **Expert-crafted prompting (few-shot CoT).** An expert-augmented ceiling using the task description, "Let's think step by step," and three chain-of-thought exemplars written by the BBH benchmark authors (Suzgun et al., 2023); unlike MPIR, this approach is not scalable and requires substantial human effort.

Together, these five conditions span a spectrum from no automation and no heuristics (manual zero-shot CoT) to full automation with heuristics but no scalability ceiling on human effort (expert-crafted few-shot CoT), with the three APO conditions and their MPIR-refined counterparts occupying the space between. This design lets Section 5 attribute MPIR's gains specifically to its rubric-guided refinement stage rather than to confounds that a narrower comparison would leave unaddressed: the free rewrite baseline controls for GPT-4o's general rewriting capability independent of any rubric; the extended APO baselines control for whether the effect is specific to PromptWizard's particular prompt structure; and the two reference points bound the range of achievable accuracy from unassisted manual prompting to unconstrained expert effort, giving Section 5.2.6 a concrete ceiling against which to interpret the remaining gap.

4.4. 4.4. Evaluation Metric

Performance is measured with accuracy: the proportion of model outputs matching ground-truth labels on the full test set. For Ntest examples with inputs xj, ground-truth labels yj, and predictions $\hat{y}$j, accuracy is (1/Ntest) Σ 1[$\hat{y}$j = yj], the average indicator of a correct prediction.

Accuracy was chosen over softer metrics such as partial-credit scoring or embedding-based similarity because every BBH task has a single, unambiguous correct answer drawn from a small option set or a short closed-form response (Section 4.1), making exact-match accuracy both well-defined and directly comparable to the original BBH benchmark paper and to the PromptWizard, APE, and ProTeGi baselines, all of which report accuracy on the same tasks. This choice has a corresponding limitation, already noted in Section 6.3: accuracy treats every incorrect answer identically regardless of how close the underlying reasoning came to correct, so it cannot by itself capture the reasoning-clarity improvements documented qualitatively in Section 5.2.4. Predicted answers are extracted programmatically from each response using the delimiter tags specified in every prompt variant (<ANS_START> and <ANS_END>, illustrated throughout Appendix A), rather than by free-form parsing of the full response text; this delimiter convention, itself one instantiation of the Output Format with Examples criterion in Section 3.3, is what makes automated accuracy scoring reliable across several thousand model responses without manual grading.

Table 2
Hyperparameter settings of PromptWizard.

Hyperparameter	Description	Default
mutate_refine_rounds	Rounds of MutateComponent followed by refinement over the best prompt generated so far.	3
mutate_rounds	Number of times MutateComponent is called.	3
style_variation	Variations MutateComponent generates per call (one per thinking style).	5
min_example_correct_count	Minimum questions ScoringComponent must answer correctly to qualify a prompt.	3
max_example_count	Maximum attempts/questions given to ScoringComponent.	6
max_seq_iter	Rounds of calls to CritiqueComponent.	3
few_shot_count	Total few-shot examples included in the prompt.	3

4.5. 4.5. Implementation Details

GPT-3.5-turbo (OpenAI, 2025b) serves as the target model for all benchmark tasks, both as the backbone of the PromptWizard baseline and for MPIR's validation stage, ensuring a consistent evaluation environment. MPIR's meta-prompting stages—evaluation and refinement—use the more capable GPT-4o (OpenAI, 2025c) to provide expert-level feedback.

PromptWizard randomly selects 25 training examples for prompt optimization and in-context example construction, with the remainder reserved for testing; MPIR uses the same 25 examples during validation to keep the comparison fair, with the remaining data again serving as the test set. Refinement runs for 7 rounds, balancing improvement opportunity against computational cost: each round issues 2 meta-prompting calls (evaluation and refinement, on GPT-4o) plus one validation call per held-out example (on GPT-3.5-turbo), so MPIR adds on the order of 200 additional API calls per task on top of the underlying APO method's own optimization cost—a real overhead that should be weighed against its per-task accuracy gains in latency- or cost-sensitive deployments. All models are accessed via API with temperature fixed at 0. Full PromptWizard hyperparameters are reported in Table 2 to support reproducibility. The implementation of MPIR is publicly available at `https://github.com/millennials92/meta_prompted_instruction_refinement`.

5. 5. Results and Analysis

5.1. 5.1. Results

Table 3 reports accuracy on the full test sets of the 23 BBH tasks (Section 4.1).

Table 4 reports accuracy before and after MPIR refinement across two further APO methods, Iterative APE and ProTeGi, on the full set of 23 BBH tasks. The tasks where MPIR moves accuracy in the same direction under all three methods most strongly are hyperbaton and ruin_names (consistently positive) and geometric_shapes and tracking_shuffled_objects (consistently negative).

Beyond aggregate performance, concrete examples illustrate where MPIR improves on baseline APO methods. In one penguins_in_a_table case (Figure 6), PromptWizard incorrectly counts two penguins younger than 8 years old, mistakenly including an 8-year-old penguin, while MPIR's refinement produces a clearer reasoning chain that correctly counts only one qualifying penguin—illustrating how rubric-driven refinement can eliminate subtle reasoning errors by enforcing stricter interpretation of task rules.

5.2. 5.2. Analysis

5.2.1. 5.2.1. Overall Performance Improvements of MPIR

Table 3 shows a directionally positive but not conclusively significant improvement over PromptWizard: MPIR reaches an average accuracy of 64.37%, versus 62.39% for PromptWizard, a difference of 1.97 percentage points, with a 95% bootstrap confidence interval of $[-0.46, 4.70]$ (10,000 resamples over the 23 tasks) that includes zero. Because the 23 tasks form matched pairs of heterogeneous difficulty rather than an unpaired sample, we follow standard practice for this design (Demšar, 2006; Dror, Baumer, Shlomov and Reichart, 2018) and supplement the bootstrap CI with three paired analyses of the same 23 task-level differences. A paired Wilcoxon signed-rank test is not significant ($W = 83.0$, $p = 0.158$), and a two-sided exact sign test on the win/loss count (16 wins, 6 losses, 1 tie) is borderline

Table 3
Accuracy (%) across 23 BBH tasks.

Task	Manual	PromptWizard	Free Rewrite	MPIR	Expert-cra
hyperbaton	74.6	62.2	68.44	84.0	84.4
disambiguation_qa	55.1	61.7	44.44	62.2	69.3
causal_judgement	53.7	59.3	59.88	56.8	59.9
date_understanding	55.6	71.1	73.33	74.2	80.0
penguins_in_a_table	45.5	75.0	75.21	82.6	74.4
boolean_expression	81.8	92.0	89.78	94.2	91.6
object_counting	59.1	91.5	85.78	92.4	88.0
word_sorting	82.6	80.0	81.78	81.3	65.3
logical_deduction	45.3	44.9	39.11	50.8	59.4
salient_translation_error_detection	44.0	50.7	49.78	52.0	54.7
geometric_shapes	29.3	57.8	52.89	49.3	64.4
snarks	45.0	63.3	59.48	65.4	72.5
temporal_sequences	33.3	44.4	44.44	38.2	17.3
web_of_lies	47.5	52.4	52.89	52.9	80.9
navigate	49.3	65.3	74.67	68.0	93.8
reasoning_about_colored_objects	54.2	56.0	53.33	68.4	83.1
sports_understanding	78.2	79.6	89.33	86.2	93.8
multistep_arithmetic_two	33.3	51.1	52.00	51.5	84.9
ruin_names	54.7	66.2	56.00	72.0	69.3
movie_recommendation	66.2	73.3	48.00	66.7	79.1
formal_fallacies	53.8	53.3	31.11	53.3	51.6
dyck_languages	1.7	14.2	14.67	12.9	16.9
tracking_shuffled_objects	22.4	69.8	18.07	65.2	63.8
Average	**50.7**	**62.39**	**57.15**	**64.37**	**69.5**

($p = 0.052$; one-sided $p = 0.026$). The paired effect size is small-to-moderate (Cohen's $dz = 0.30$). We report all three rather than selecting the most favorable one: together they indicate a real but modest and not fully conclusive effect, consistent with typical statistical power at this task count (Card, Henderson, Khandelwal, Jia, Mahowald and Jurafsky, 2020), rather than an established, strongly significant improvement. The more robust evidence for MPIR is qualitative and structural—its consistency across individual tasks (16 of 23, with several gains of 10-20 points), across three different APO frameworks (Section 5.2.2), and across two model families (Section 5.2.3)—together with the free rewrite baseline, which reaches only 57.15% on average, well below both PromptWizard and MPIR. Together, these results indicate that MPIR's per-task gains are attributable to its structured seven-criteria rubric rather than simply to GPT-4o's general rewriting ability, even though the pooled average improvement should be read as a promising, consistent trend rather than a statistically confirmed effect; Section 6.1 revisits this as a threat to validity.

5.2.2. 5.2.2. Evaluating MPIR Across Multiple APO Frameworks

Table 4 shows that MPIR consistently improves average accuracy across APO frameworks: Iterative APE from 70.16% to 74.02%, ProTeGi from 72.81% to 74.26%, and PromptWizard from 62.39% to 64.37%. Although the magnitude of improvement varies across methods and tasks, the consistently positive trend supports MPIR functioning as a refinement layer across different APO frameworks rather than one tailored to PromptWizard-specific prompt structures.

5.2.3. 5.2.3. Cross-Model Generalization

To test MPIR under a different model family, we repeated the experiment with Gemini 3.5 Flash-Lite (Google DeepMind, 2026) as both the target model and the meta-prompting model (Table 5). The baseline already achieves a high average accuracy of 92%, leaving limited room for improvement, and MPIR maintains the same rounded average after refinement. The most noticeable gains occur where baseline accuracy was around 70%: causal_judgement improves from 70% to 75%, disambiguation_qa from 77% to 80%, and dyck_languages from 72% to 75%. Tasks already near-perfect before refinement, such as sports_understanding and multistep_arithmetic_two, show little room

Table 4
Accuracy (%) before and after MPIR refinement, for three APO methods, across all 23 BBH tasks.

Task	APE (before)	ProTeGi (before)	PromptWizard (before)	APE (after)	ProTeGi (after)	PromptWizard (after)
hyperbaton	82.67	80.89	62.2	85.78	88.89	84.0
disambiguation_qa	69.78	73.78	61.7	68.00	67.56	62.2
causal_judgement	58.02	61.11	59.3	58.64	58.64	56.8
date_understanding	87.56	88.00	71.1	84.44	85.78	74.2
penguins_in_a_table	80.99	79.34	75.0	84.30	76.86	82.6
boolean_expression	90.67	90.67	92.0	93.78	92.89	94.2
object_counting	92.00	88.44	91.5	93.33	93.33	92.4
word_sorting	69.33	68.44	80.0	80.00	66.67	81.3
logical_deduction	61.93	60.74	44.9	59.70	59.26	50.83
salient_translation_error_detection	55.11	52.89	50.7	54.67	49.33	52.0
geometric_shapes	62.67	61.78	57.8	61.33	60.89	49.3
snarks	61.44	75.82	63.3	68.63	73.20	65.4
temporal_sequences	26.67	84.00	44.4	86.67	86.22	38.2
web_of_lies	80.00	81.33	52.4	73.33	80.89	52.9
navigate	95.56	95.56	65.3	92.89	97.78	68.0
reasoning_about_colored_objects	82.67	82.67	56.0	82.67	83.56	68.4
sports_understanding	95.11	93.33	79.6	96.44	94.67	86.2
multistep_arithmetic_two	86.22	83.56	51.1	84.89	82.67	51.5
ruin_names	58.67	58.67	66.2	68.44	77.78	72.0
movie_recommendation	76.89	76.00	73.3	79.11	80.89	66.7
formal_fallacies	52.44	55.11	53.3	52.89	56.89	53.3
dyck_languages	24.44	20.89	14.2	34.22	32.44	12.9
tracking_shuffled_objects	62.96	61.63	69.8	58.22	60.89	65.5
Average (23 tasks)	**70.16**	**72.81**	**62.39**	**74.02**	**74.26**	**64.37**

for further gains and occasionally a slight decline—suggesting MPIR is most effective when the baseline prompt still has meaningful reasoning weaknesses to correct.

5.2.4. 5.2.4. Clarity, Structure, and Task Difficulty

MPIR also improves interpretability by embedding human-inspired structure, clarifying reasoning context, and filtering out details that could distract the model. In hyperbaton, PromptWizard's baseline prompt mixed background explanation with task directives in a single block, whereas MPIR separated it into a context section and a numbered instruction section, helping the model distinguish framing from required actions; similar edits elsewhere (e.g., recasting a vague instruction in web_of_lies into an explicit role-and-goal statement) removed distracting, task-irrelevant content that had been reducing output quality.

Across Tables 3-5, MPIR's strongest and most consistent gains occur on tasks governed by explicit structural or rule-based reasoning—hyperbaton, object_counting, boolean_expression, and temporal_sequences all improve repeatedly across APO frameworks and model families (e.g., hyperbaton rises from 62.2% to 84.0% under PromptWizard). This is consistent with prior evidence that CoT prompting helps models decompose complex problems into intermediate steps (Wei et al., 2022) and that step-back reasoning helps LLMs derive first principles before solving a task (Zheng et al., 2024). MPIR is comparatively less effective on tasks requiring precise symbolic manipulation, spatial abstraction, or extended sequential reasoning—geometric_shapes, tracking_shuffled_objects, and logical_deduction show smaller, less stable gains or occasional regressions (e.g., geometric_shapes falls from 57.8% to 49.3% under PromptWizard). This pattern matches prior findings that coherent, relevant reasoning steps matter more than reasoning length (Wang et al., 2023a), and that LLMs remain highly sensitive to subtle prompt-phrasing changes (Errica et al., 2025), which can disrupt the precise consistency these tasks demand.

5.2.5. 5.2.5. Detailed Failure Mode Analysis

Appendix C.1 illustrates the symbolic-tracking failure pattern concretely on a tracking_shuffled_objects_five_objects example. Both PromptWizard and MPIR correctly identify the sequence of pairwise swaps described in the question, and both attempt to apply them in order—the failure is not in understanding the task, but in maintaining a consistent internal representation of state across several sequential updates. PromptWizard's reasoning trace re-derives the full mapping after each swap, which is verbose but self-correcting: an error at one step does not necessarily propagate, because the next step re-examines all positions rather than incrementally updating a single pair. MPIR's refined prompt, by contrast, produces a more compact, incrementally updated trace consistent with the Instruction and Separation and Guided Chain of Thought criteria it was optimized toward—but that same compactness means a single

Table 5
Cross-model evaluation of MPIR on Gemini 3.5 Flash-Lite across all 23 BBH tasks.

Task	PromptWizard (%)	MPIR (%)	Change (%)
hyperbaton	100	100	0
disambiguation_qa	77	80	+3
causal_judgement	70	75	+5
date_understanding	95	96	+1
penguins_in_a_table	100	98	−2
boolean_expressions	100	100	0
object_counting	100	100	0
word_sorting	92	92	0
logical_deduction	99	96	−3
salient_translation_error_detection	75	74	−1
geometric_shapes	86	83	−3
snarks	90	90	0
temporal_sequences	100	100	0
web_of_lies	100	100	0
navigate	100	99	−1
reasoning_about_colored_objects	100	100	0
sports_understanding	91	86	−5
multistep_arithmetic_two	97	94	−4
ruin_names	88	87	−1
movie_recommendation	95	96	+1
formal_fallacies	99	99	0
dyck_languages	72	75	+3
tracking_shuffled_objects	100	100	0
Average (23 tasks)	**92**	**92**	**0**

misapplied swap is never re-checked against the full state, and the error persists to the final answer. This is a case where two criteria that are individually beneficial for the structured, rule-based tasks discussed above (Section 5.2.4) interact unfavorably with a task that specifically requires redundant self-checking rather than compactness.

A related pattern appears in logical_deduction and geometric_shapes, where the ablation in Table 7 shows Instruction and Separation (C4) as the single most load-bearing criterion overall, yet Table 3 and Table 4 show these same two tasks among MPIR's weakest relative to PromptWizard. Instruction and Separation improves clarity by isolating the task statement from surrounding context, but in these two task types the "context" being separated out often contains constraints the reasoning process must repeatedly refer back to—for example, an ordering constraint in logical_deduction or a shape's coordinate definition in geometric_shapes—so isolating it into a labeled context block does not reduce the cognitive load of the task the way it does for tasks where context is genuinely background rather than an active constraint. This suggests the seven criteria are not uniformly beneficial across task types, but rather beneficial conditional on the type of reasoning error a task is prone to—a distinction the current rubric does not represent explicitly, and which an instance-adaptive rubric (Section 7) could in principle capture.

5.2.6. 5.2.6. Remaining Gap to Expert-Crafted Prompting

Despite strong gains over automated baselines, MPIR still falls short of expert-crafted prompting on average (64.4% versus 69.5%). The gap concentrates in a small set of tasks—navigate, reasoning_about_colored_objects, web_of_lies, and multistep_arithmetic_two (Table 6)—where expert examples provide richer reasoning traces and more explicit

Table 6
Tasks where MPIR underperforms expert-crafted prompting.

Task	MPIR (%)	Expert (%)
navigate	68.0	93.8
reasoning_about_colored_objects	68.4	83.1
web_of_lies	52.9	80.9
multistep_arithmetic_two	51.5	84.9

Table 7
Effect of removing individual rubric criteria across five BBH tasks (accuracy, %). C1=Role Prompting, C2=Step Back, C3=Guided CoT, C4=Instruction & Separation, C5=Output Format, C6=Worked Reasoning, C7=Conclusion.

Task	ALL	C1	C2	C3	C4	C5	C6	C7
hyperbaton	84.0	56.0	83.1	67.6	56.4	59.1	59.6	79.1
penguins_in_a_table	82.6	76.0	67.8	70.2	45.5	73.6	79.3	75.2
ruin_names	72.0	52.4	51.1	50.2	54.7	67.1	60.4	57.3
object_counting	92.4	90.2	91.6	88.4	91.6	92.0	88.4	92.0
reasoning_about_colored_objects	68.4	59.1	52.0	68.9	79.6	62.7	60.9	69.8
Average	**79.9**	**67.0**	**69.0**	**69.0**	**66.0**	**71.0**	**70.0**	**75.0**

state tracking. Because MPIR's examples are inherited from the PromptWizard baseline rather than newly generated, they reflect the same limitations present in PromptWizard's own outputs: MPIR's refinements are structurally consistent but tend to oversimplify example reasoning relative to expert-written traces.

5.3. 5.3. Ablation Studies

5.3.1. 5.3.1. Which Rubric Criteria Matter Most

To assess each rubric criterion's contribution, we ran an ablation on five representative BBH tasks, removing one criterion at a time (Table 7). Removing any single criterion reduces average accuracy from the full rubric's 79.9% to between 66.0% and 75.0%, so every criterion plays a meaningful role, though to different degrees. Four criteria prove particularly critical—Instruction & Separation (C4), Role Prompting (C1), Guided Chain of Thought (C3), and Step Back (C2)—each reducing average accuracy by over 11 points when removed, underscoring their role in structuring reasoning and reducing ambiguity. Removing Worked Reasoning in Examples (C6) produces a moderate but consistent drop, suggesting that step-by-step demonstrations reinforce structured reasoning patterns, whereas removing Output Format Specification (C5) or Conclusion (C7) causes only minor drops, indicating these criteria mainly enhance clarity rather than drive task accuracy. Overall, heuristics that organize reasoning and contextual framing appear to contribute most to refinement performance on these benchmark tasks.

5.3.2. 5.3.2. Heuristic Rubric vs. Generic Rubric

To test whether the rubric's specificity matters, we compared the full seven-criteria rubric against a generic rubric using broad criteria such as clarity, structure, and effectiveness, on the same five tasks, holding refinement and validation fixed (Table 8). The full rubric outperforms the generic one on four of five tasks, with average accuracy dropping from 79.9% to 72.2% under the generic rubric—an 8-point gap confirming that the seven-criteria rubric's specificity, not just the act of evaluation and refinement, drives MPIR's effectiveness. The heuristic rubric enforces step-by-step reasoning, context separation, and structured outputs, whereas the generic rubric's vaguer feedback often fails to correct task-specific weaknesses.

Table 8
Effect of the generic rubric and of removing prompt-effectiveness validation, each measured against the same full-MPIR baseline, across five BBH tasks (accuracy, %).

Task	MPIR (full)	Generic rubric	No validation
hyperbaton	84.0	66.2	37.8
penguins_in_a_table	82.6	78.5	67.8
ruin_names	72.0	66.2	52.0
object_counting	92.4	92.9	91.6
reasoning_about_colored_objects	68.4	57.3	60.4
Average	**79.9**	**72.2**	**61.9**

5.3.3. 5.3.3. Importance of Prompt Effectiveness Validation

Finally, we tested whether the validation stage itself is necessary by comparing the full framework (seven evaluation-refinement-validation cycles) against a validation-ablated variant that selects a prompt after a single evaluation-refinement cycle with no empirical testing, again on five representative tasks (Table 8). Removing validation drops average accuracy from 79.9% to 61.9%, an 18-point decline showing that rubric alignment alone does not guarantee performance: an unvalidated prompt may look well structured yet perform worse empirically. Validation anchors MPIR's refinements in measured accuracy, ensuring gains are both real and task-oriented rather than purely heuristic.

5.4. 5.4. Revisiting the Research Objectives

Section 1 set out four objectives for this study; we revisit each in light of the results above.

- **Objective 1 (a structured rubric for prompt evaluation):** achieved. Section 3.3 formalizes seven manual prompting heuristics into an explicit, criterion-by-criterion rubric, and Section 5.3.1 shows the criteria are not redundant with one another—removing any single one measurably reduces accuracy, with Instruction and Separation, Role Prompting, Guided Chain of Thought, and Step Back identified as the most load-bearing.
- **Objective 2 (whether meta-prompted refinement improves APO-generated prompts):** mixed evidence. MPIR improves PromptWizard on 16 of 23 BBH tasks, but none of the three paired tests in Section 5.2.1 (Section 6.1) confirm the pooled average improvement at conventional significance thresholds—this null pooled result stands as reported rather than being explained away, and the per-task win rate is offered as complementary evidence, not a substitute for it.
- **Objective 3 (generalizability and modularity across APO frameworks and models):** achieved within the tested scope. Section 5.2.2 shows consistent improvement when MPIR is applied to two additional APO methods beyond PromptWizard, and Section 5.2.3 shows the same pattern holds under a different model family; Section 6.2 notes this evidence does not extend to benchmark families beyond BBH or to task types such as open-ended generation.
- **Objective 4 (how individual heuristics contribute to performance):** achieved. The ablation in Section 5.3.1 quantifies each criterion's individual contribution, and the comparison with a generic rubric in Section 5.3.2 further shows that the specific content of the seven criteria, not merely the act of iterative evaluation and refinement, drives the observed gains.

5.5. 5.5. Practical Implications for Practitioners

The results in this section suggest concrete guidance for a practitioner deciding whether to add MPIR on top of an existing APO pipeline. First, MPIR is best suited to settings where the target task involves explicit structural or rule-based reasoning—sorting, counting, boolean evaluation, temporal ordering—since Section 5.2.4 shows these are exactly the tasks where rubric-guided refinement most reliably helps; for tasks requiring precise symbolic manipulation

or long sequential state tracking, the expected gain is smaller and occasionally negative, and a practitioner in that regime may be better served by investing directly in expert-crafted examples (Section 5.2.6) than in an automated refinement layer. Second, MPIR is most useful when the upstream APO system has room to improve: Section 5.2.3 shows that once baseline accuracy is already near-ceiling, MPIR's rubric-guided edits offer little additional benefit and can occasionally introduce small regressions, so it is not necessary to apply MPIR to prompts that are already performing well. Third, the per-task overhead is small and bounded—on the order of 200 additional API calls per task for the configuration used in this paper (Section 4.5)—so the primary cost-benefit question is whether the target application can tolerate that one-time refinement cost in exchange for a modest but consistent accuracy improvement, rather than whether the technique is computationally prohibitive in absolute terms. Fourth, because MPIR requires no retraining, gradient access, or bespoke search infrastructure, it is straightforward to retrofit onto an already-deployed APO pipeline without modifying that pipeline's own code, which may make it attractive in production settings where introducing a new training or search dependency is undesirable even when a more powerful but heavier method such as GEPA (Agrawal et al., 2026) might offer a larger expected gain.

Beyond the BBH benchmark used for evaluation, the underlying pattern MPIR targets—an APO system produces a serviceable but imperfect prompt, and a small set of interpretable heuristics can catch specific, nameable weaknesses in it—plausibly recurs in several applied settings that share BBH's combination of structured tasks and automated prompt tuning. Customer-support and helpdesk deflection systems, for instance, often rely on an APO-tuned prompt to classify or answer routine queries; the kinds of errors documented in Section 5.2.4, where a baseline prompt buries the actual instruction inside irrelevant context, are directly analogous to a support prompt that mixes company-specific background with the actual classification task. Educational tutoring systems that generate step-by-step explanations depend heavily on the Guided Chain of Thought and Worked Reasoning in Examples criteria identified as most load-bearing in the ablation (Section 5.3.1), suggesting that a rubric-guided refinement pass could be particularly relevant there. Retrieval-augmented enterprise assistants, which typically combine a fixed instruction template with dynamically retrieved context, could apply MPIR's Instruction and Separation criterion specifically to keep the static instruction distinguishable from retrieved content that changes per query. We have not evaluated MPIR in any of these settings, and Section 6.2 already notes that generalization beyond BBH-style reasoning tasks is untested; we raise them here as plausible directions for the broader applied evaluation Section 7 calls for, in keeping with this journal's focus on actionable applications of AI research.

6. 6. Threats to Validity

This section organizes the study's limitations, several already introduced alongside individual results, into the standard internal, external, and construct validity framing used in empirical software and machine learning research, so that their scope and interaction are considered together rather than piecemeal.

6.1. 6.1. Internal Validity

The central internal-validity concern is statistical: the average improvement over PromptWizard (1.97 points) is not conclusively significant by any of the three paired tests reported in Section 5.2.1, and all results come from a single run per condition at fixed temperature 0 rather than repeated trials with varied seeds or example orderings. A second concern is evaluator bias: MPIR's meta-prompting stages and the free rewrite baseline both use GPT-4o as judge and rewriter respectively, so any systematic preference GPT-4o has for its own stylistic conventions could inflate MPIR's apparent quality independently of downstream task accuracy; the validation stage (Section 3.5) partially controls for this by requiring an accuracy gain on GPT-3.5-turbo, a different model, before a candidate is accepted, but cannot rule it out entirely. A third concern is that MPIR reuses PromptWizard's own 25 optimization examples during validation (Section 4.5) to keep the comparison fair; this is a deliberate design choice rather than an oversight, but it does mean the validation signal and the optimization signal are drawn from overlapping data.

6.2. 6.2. External Validity

Generalizability is bounded along three axes tested in this paper and open along others not tested. Within the tested axes, MPIR's benefit generalizes across three APO backbones (PromptWizard, Iterative APE, ProTeGi; Section 5.2.2) and across two model families (GPT-3.5-turbo and Gemini 3.5 Flash-Lite; Section 5.2.3), which is meaningfully broader evidence than a single-backbone, single-model result would provide. Outside the tested axes, all results are on Big-Bench Hard, a reasoning-and-instruction-following benchmark; performance on other task families such as

open-ended generation, retrieval-augmented question answering, or code generation is untested. BBH also remains meaningfully unsaturated only for the cheap-tier target models used here; results might differ for frontier-tier target models, where BBH is reported to be substantially saturated (Suzgun et al., 2023). Finally, GPT-3.5-turbo, the primary target model, is scheduled for API retirement on October 23, 2026 (OpenAI, 2026), so external validity for future reproductions will depend on its successor behaving comparably.

6.3. 6.3. Construct Validity

The seven-criteria rubric (Section 3.3) is the paper's central construct, and it was developed in proximity to the BBH task family it is evaluated on, drawing on general prompting literature (Schulhoff et al., 2024; Sahoo et al., 2024; Li et al., 2025) and industry guidance (Boonstra, 2025; Anthropic, 2025; OpenAI, 2025a; Microsoft, 2025) but refined through iterative experimentation on these same tasks. This creates a risk of construct dependence: the rubric may capture properties that happen to matter for BBH-style reasoning tasks specifically, rather than prompt quality in a domain-independent sense. The ablation in Section 5.3.1 shows the seven criteria are not equally load-bearing, which is consistent with either a well-differentiated construct or with some criteria being closer proxies for BBH performance than others; the study cannot distinguish between these two explanations without evaluating the rubric on an independent benchmark family, which Section 7 lists as future work. A second construct concern is the evaluation metric itself: accuracy against a single ground-truth answer treats near-miss and far-miss errors identically, which may understate MPIR's qualitative improvements in reasoning clarity documented in Section 5.2.4.

7. 7. Conclusion

This paper introduced Meta-Prompted Instruction Refinement (MPIR), a lightweight, model-agnostic framework that integrates manual prompting heuristics into automatic prompt optimization through a structured cycle of evaluation, refinement, and validation. It asks whether human-inspired prompting heuristics can be systematically integrated into APO systems to improve prompt quality while preserving the scalability of automated optimization, and does so deliberately without the additional training, search infrastructure, or per-task tuning that increasingly elaborate 2025-2026 prompt-optimization methods require (Section 2.5). Experiments on Big-Bench Hard show that rubric-guided meta-prompt refinement can improve APO-generated prompts across multiple tasks and APO frameworks, and that MPIR is particularly effective for structured, rule-based reasoning tasks, where heuristic-guided refinement yields clearer and more reliable reasoning.

This study also has limitations, discussed in depth as threats to validity in Section 6: the average gain over the PromptWizard baseline is modest and not conclusively significant by any single test (Section 6.1); results come from a single run per condition rather than repeated trials with varying seeds, a gap common across the prompt-optimization literature but one that recent evidence on the brittleness of LLM evaluations makes worth closing (Section 6.1); generalization beyond the three tested APO frameworks, two tested model families, and the BBH benchmark itself remains open (Section 6.2); and the seven-criteria rubric was developed in proximity to the BBH tasks it is evaluated on, which may introduce construct dependence (Section 6.3). We summarize the strongest, most defensible evidence for MPIR's contribution as qualitative and structural—consistency across individual tasks, APO frameworks, and model families—rather than the pooled average improvement alone.

Future work could extend MPIR along several directions. Repeated-trial evaluation with multiple seeds and example orderings would directly address the internal-validity concern in Section 6.1 by separating run-to-run optimization variance from genuine task-level effects, and would let the current bootstrap and paired tests be supplemented with variance estimates that account for both sources of noise simultaneously. Independent construct validation—deriving the seven-criteria rubric, or a variant of it, on one benchmark family and evaluating its transfer to a distinct one, such as open-ended generation or retrieval-augmented question answering—would directly test the construct-dependence concern raised in Section 6.3. Adaptive or instance-specific rubrics, in the spirit of the learned-rubric direction noted in Section 2.5, could replace MPIR's fixed seven criteria with criteria selected or weighted per task, potentially recovering some of the benefit of heavier methods such as GEPA while retaining MPIR's lower training and infrastructure cost. Applying MPIR to the frontier-model regime, where BBH itself is largely saturated (Suzgun et al., 2023), would require pairing it with a harder successor benchmark such as BIG-Bench Extra Hard rather than BBH directly. Applied evaluation beyond BBH-style reasoning tasks—in customer-support and helpdesk deflection prompts, tutoring systems that generate worked explanations, and retrieval-augmented enterprise assistants, the three settings identified as plausible fits in Section 5.5—would test whether the gains reported here transfer to the

applied AI systems this journal's scope emphasizes. Finally, combining MPIR with complementary approaches such as symbolic reasoning modules, retrieval augmentation, self-consistency decoding (Wang et al., 2023b), or automated methods for learning prompting heuristics directly from empirical performance data are all directions that could be pursued independently of one another, since none of them are mutually exclusive with the rubric-guided refinement loop introduced here.

8. Acknowledgement

The authors thank the reviewers for their constructive comments on earlier drafts of this manuscript.

9. Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

10. Ethical Statement

This study did not involve human participants, human data, or animal subjects, and did not require ethical approval.

11. Declaration of Competing Interest

The authors declare that they have no conflicts of interest.

12. Data Availability Statement

The datasets used in this study are drawn from the publicly available Big-Bench Hard benchmark. Code, prompts, and experimental results are openly available at `https://github.com/millennials92/meta_prompted_instruction_refinement`.

13. Author Contribution Statement

Linh Nguyen: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Writing - original draft, Visualization. Quang-Vinh Dang: Conceptualization, Methodology, Supervision, Writing - review & editing, Project administration. Minh Ngoc Dinh: Methodology, Validation, Writing - review & editing. Thuy Nguyen: Investigation, Validation, Writing - review & editing.

14. Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

Generative AI tools were used solely to improve the grammar and readability of the text. All ideas, analyses, and conclusions are solely those of the authors. An AI-assisted coding tool was used to generate initial code snippets during development; all code was reviewed, verified, and adapted by the authors.

A. Prompt Examples

A.1. A.1. Prompts for the penguins_in_a_table Task

To make the qualitative discussion in Section 5.2.4 and the case study in Figure 6 self-contained, this appendix reproduces the full prompt text used at each stage of the pipeline for one representative task, penguins_in_a_table, exactly as issued to the model.

A.2. A.2. Few-Shot Examples for the Navigate Task

The navigate task asks whether a sequence of turns and steps returns to the starting point. Figure A.5 and Figure A.6 contrast the expert-crafted worked example used as the Section 4.3.3 reference point with the corresponding MPIR-refined example, illustrating the gap discussed in Section 5.2.6: MPIR's example, inherited from PromptWizard, tracks state as a bulleted sequence of moves without explicit coordinates, whereas the expert-crafted example maintains an explicit running position.

Table 9
Complete experimental protocol summary.

Setting	Value
Benchmark	Big-Bench Hard, 23 tasks (Suzgun et al., 2023)
Primary target model	GPT-3.5-turbo, temperature 0
Meta-prompting model (evaluation and refinement)	GPT-4o, temperature 0
Cross-model target/meta model	Gemini 3.5 Flash-Lite, temperature 0
Primary APO baseline	PromptWizard (unmodified hyperparameters, Table 2)
Extended APO baselines	Iterative APE; ProTeGi
Training/optimization examples per task	25, randomly sampled
Held-out test examples per task	Remainder of each task's example set
MPIR refinement rounds (N)	7
Rubric criteria evaluated per round	7 (Section 3.3)
Candidate selection rule	Highest held-out accuracy across all N rounds
Ablation task subset (Sections 5.3.1-5.3.3)	hyperbaton, penguins_in_a_table, ruin_names, object_counting, reasoning_about_colored_objects
Significance tests reported	Bootstrap CI (10,000 resamples); paired Wilcoxon signed-rank; exact sign test; Cohen's dz
Runs per condition	1 (single run at temperature 0; see Section 6.1)

B. Generic Rubric Baseline Prompt

Section 5.3.2 compares MPIR's seven-criteria rubric against a generic rubric that evaluates prompts on broad, undifferentiated aspects of quality rather than the specific heuristics in Section 3.3. The full meta-prompt used for that generic-rubric evaluation stage is reproduced below; the refinement and validation stages are otherwise identical to MPIR's own (Sections 3.4-3.5).

C. Additional Case Study

Section 5.1 and Figure 6 present a case in which MPIR corrects a PromptWizard reasoning error. Not every case favors MPIR: Figure C.1 shows a tracking_shuffled_objects_five_objects example where PromptWizard and MPIR follow the same sequence of swaps but diverge in intermediate bookkeeping, with PromptWizard reaching the correct answer and MPIR misapplying one swap. This example is representative of the symbolic-tracking weakness discussed in Section 5.2.4.

D. Free Rewrite Baseline Prompt

Section 4.3.1 introduces a free rewrite baseline that isolates MPIR's rubric-guided evaluation from GPT-4o's general rewriting ability. The full meta-prompt used to produce that baseline is reproduced below.

E. Reproducibility and Experimental Protocol

This appendix consolidates the settings scattered across Sections 3-5 into a single reference for reproduction, following the convention that empirical machine learning papers report a complete, self-contained protocol rather than requiring a reader to reassemble it from prose.

All code, configuration files, prompt templates, and per-task raw results referenced throughout this paper are available at the project repository (Section 4.5), including the exact YAML configuration files for PromptWizard and MPIR, the rubric prompt templates reproduced in Figures 2 and 3, and the full 23-task results underlying every averaged or condensed figure reported above.

References

Agarwal, E., Magazine, R., Singh, J., Dani, V., Ganu, T., Nambi, A., 2025. Promptwizard: Optimizing prompts via task-aware, feedback-driven self-evolution, in: Findings of the Association for Computational Linguistics: ACL 2025, pp. 19974–20003.

Agrawal, L.A., Tan, S., Soylu, D., Ziems, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M.J., Jiang, M., Potts, C., Sen, K., Dimakis, A.G., Stoica, I., Klein, D., Zaharia, M., Khattab, O., 2026. Gepa: Reflective prompt evolution can outperform reinforcement learning, in: Proceedings of the International Conference on Learning Representations.

Anthropic, 2025. Prompt engineering overview. Anthropic Documentation.

Boonstra, L., 2025. Prompt engineering (kaggle whitepaper). Google.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Amodei, D., et al., 2020. Language models are few-shot learners, in: Proceedings of the 34th International Conference on Neural Information Processing Systems (Article 159).

Card, D., Henderson, P., Khandelwal, U., Jia, R., Mahowald, K., Jurafsky, D., 2020. With little power comes great responsibility, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, pp. 9263–9274.

Chakma, A., Surdeanu, M., Blanco, E., 2026. Two-stage prompt optimization for few-shot relation extraction: From reasoning-guided search to gradient-guided refinement. arXiv:2606.29639.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J., 2021. Training verifiers to solve math word problems. arXiv:2110.14168.

Coda-Forno, J., Binz, M., Wang, J.X., Schulz, E., 2024. Cogbench: A large language model walks into a psychology lab, in: Proceedings of the 41st International Conference on Machine Learning.

Colombo, P., Pires, T.P., Boudiaf, M., Culver, D., Melo, R., Corro, C., Martins, A.F.T., Esposito, F., Raposo, V.L., Morgado, S., Desa, M., 2024. Saullm-7b: A pioneering large language model for law. arXiv:2403.03883.

Cui, W., Zhang, J., Li, Z., Sun, H., Lopez, D., Das, K., Malin, B.A., Kumar, S., 2025. Heuristic-based search algorithm in automatic instruction-focused prompt optimization: A survey, in: Findings of the Association for Computational Linguistics: ACL 2025, pp. 22093–22111.

Demšar, J., 2006. Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research 7, 1–30.

Dror, R., Baumer, G., Shlomov, S., Reichart, R., 2018. The hitchhiker's guide to testing statistical significance in natural language processing, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1383–1392.

Errica, F., Sanvito, D., Siracusano, G., Bifulco, R., 2025. What did i do wrong? quantifying llms' sensitivity and consistency to prompt engineering, in: Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1543–1558.

Google DeepMind, 2026. Gemini 3.5 flash-lite (model documentation).

Goyal, T., Li, J.J., Durrett, G., 2023. News summarization and evaluation in the era of gpt-3. arXiv:2209.12356.

Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., Liu, G., Bian, J., Yang, Y., 2024. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers, in: The Twelfth International Conference on Learning Representations.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T.T., Moazam, H., Miller, H., Zaharia, M., Potts, C., 2024. Dspy: Compiling declarative language model calls into state-of-the-art pipelines, in: The Twelfth International Conference on Learning Representations.

Kim, J., Yang, N., Jung, K., 2024. Persona is a double-edged sword: Mitigating the negative impact of role-playing prompts in zero-shot reasoning tasks. arXiv:2408.08631.

Kojima, T., Gu, S.S., Reid, M., Matsuo, Y., Iwasawa, Y., 2022. Large language models are zero-shot reasoners, in: Proceedings of the 36th International Conference on Neural Information Processing Systems (Article 1613).

Kong, A., Zhao, S., Chen, H., Li, Q., Qin, Y., Sun, R., Zhou, X., Wang, E., Dong, X., 2024. Better zero-shot reasoning with role-play prompting, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 4099–4113.

Li, W., Wang, X., Li, W., Jin, B., 2025. A survey of automatic prompt engineering: An optimization perspective. arXiv:2502.11560.

Lin, H.C., Lubis, N., Ruppik, B.M., van Niekerk, C., Shen, C.H., Heck, M., Vukovic, R., Feng, S., Gasic, M., 2025. Prompt reinforcing for long-term planning of large language models arXiv:2510.05921.

Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P., 2024. Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics 12, 157–173.

Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G., 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. ACM Computing Surveys, 55(9), Article 195.

Liu, Y., Xu, J., Zhang, L.L., Chen, Q., Feng, X., Chen, Y., Guo, Z., Yang, Y., Cheng, P., 2025. Beyond prompt content: Enhancing llm performance via content-format integrated prompt optimization. arXiv:2502.04295.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P., 2023. Self-refine: Iterative refinement with self-feedback, in: Thirty-seventh Conference on Neural Information Processing Systems.

Mayilvaghanan, K., Nathan, V., Kumar, A., 2025. Propel: Prompt optimization with expert priors for small and medium-sized llms, in: Proceedings of the 4th Workshop on Knowledge-Augmented Methods for Natural Language Processing, pp. 272–302.

Microsoft, 2025. Prompt engineering techniques. Azure AI Foundry Documentation.

Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., Zettlemoyer, L., 2022. Rethinking the role of demonstrations: What makes in-context learning work?, in: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp. 11048–11064.

Nair, A., Banerjee, A., Mombaerts, L., Hagen, M., Borogovac, T., 2025. Tournament of prompts: Evolving llm instructions through structured debates and elo ratings, in: KDD 2025 Workshop on Prompt Optimization.

OpenAI, 2025a. Best practices for prompt engineering with the openai api. OpenAI Help Center.

OpenAI, 2025b. GPT-3.5 turbo (model documentation).

OpenAI, 2025c. Gpt-4o (model documentation).

OpenAI, 2026. Deprecations. OpenAI API documentation.

Pryzant, R., Iter, D., Li, J., Lee, Y.T., Zhu, C., Zeng, M., 2023. Automatic prompt optimization with "gradient descent" and beam search, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing.

Qin, L., Chen, Q., Feng, X., Wu, Y., Zhang, Y., Li, Y., Li, M., Che, W., Yu, P.S., 2025. Large language models meet nlp: A survey. arXiv:2405.12819.

Ramnath, K., Zhou, K., Guan, S., Mishra, S.S., Qi, X., Shen, Z., Wang, S., Woo, S., Jeoung, S., Wang, Y., Wang, H., Ding, H., Lu, Y., Xu, Z., Zhou, Y., Srinivasan, B., Yan, Q., Chen, Y., Ding, H., Xu, P., Cheong, L.L., 2025. A systematic survey of automatic prompt optimization techniques. arXiv:2502.16923.

Reynolds, L., McDonell, K., 2021. Prompt programming for large language models: Beyond the few-shot paradigm, in: Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems (Article 314).

Sahoo, P., Singh, A.K., Saha, S., Jain, V., Mondal, S., Chadha, A., 2024. A systematic survey of prompt engineering in large language models: Techniques and applications. arXiv:2402.07927.

Schulhoff, S., Ilie, M., Balepur, N., Kahadze, K., Liu, A., Si, C., Li, Y., Gupta, A., Han, H., Schulhoff, S., Dulepet, P.S., Vidyadhara, S., Ki, D., Agrawal, S., Pham, C., Kroiz, G.C., Li, F., Tao, H., Resnik, P., et al., 2024. The prompt report: A systematic survey of prompting techniques. arXiv:2406.06608.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K.R., Yao, S., 2023. Reflexion: Language agents with verbal reinforcement learning, in: Thirty-seventh Conference on Neural Information Processing Systems.

Singh, M., Yadav, V., Blanco, E., 2026. Error taxonomy-guided prompt optimization. arXiv:2602.00997.

Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H.W., Chowdhery, A., Le, Q., Chi, E., Zhou, D., Wei, J., 2023. Challenging big-bench tasks and whether chain-of-thought can solve them, in: Findings of the Association for Computational Linguistics: ACL 2023, pp. 13003–13051.

Wang, B., Min, S., Deng, X., Shen, J., Wu, Y., Zettlemoyer, L., Sun, H., 2023a. Towards understanding chain-of-thought prompting: An empirical study of what matters, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 2717–2739.

Wang, R., Mi, F., Chen, Y., Xue, B., Wang, H., Zhu, Q., Wong, K.F., Xu, R., 2024. Role prompting guided domain adaptation with general capability preserve for large language models, in: Findings of the Association for Computational Linguistics: NAACL 2024, pp. 2243–2255.

Wang, X., Wei, J., Schuurmans, D., Le, Q.V., Chi, E.H., Narang, S., Chowdhery, A., Zhou, D., 2023b. Self-consistency improves chain of thought reasoning in language models, in: The Eleventh International Conference on Learning Representations.

Wang, X., Zhou, D., 2024. Chain-of-thought reasoning without prompting, in: Proceedings of the 38th International Conference on Neural Information Processing Systems.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D., 2022. Chain-of-thought prompting elicits reasoning in large language models, in: Proceedings of the 36th International Conference on Neural Information Processing Systems (Article 1800).

Winata, G.I., Huang, L.K., Vadlamannati, S., Chandarana, Y., 2023. Multilingual few-shot learning via language model retrieval. arXiv:2306.10964.

Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q.V., Zhou, D., Chen, X., 2024a. Large language models as optimizers, in: The Twelfth International Conference on Learning Representations.

Yang, M., Li, M., Li, Y., Chen, Z., Gao, C., Zhang, J., Li, Y., Feng, F., 2024b. Dual-phase accelerated prompt optimization, in: Findings of the Association for Computational Linguistics: EMNLP 2024, pp. 12163–12173.

Ye, Q., Ahmed, M., Pryzant, R., Khani, F., 2024. Prompt engineering a prompt engineer, in: Findings of the Association for Computational Linguistics: ACL 2024, pp. 355–385.

Yu, Y., Jiang, H., Luo, X., Wu, Q., Lin, C.Y., Li, D., Yang, Y., Huang, Y., Qiu, L., 2025. Mitigate position bias in llms via scaling a single hidden states channel, in: Findings of the Association for Computational Linguistics: ACL 2025, pp. 6092–6111.

Yuksekgonul, M., Bianchi, F., Boen, J., Liu, S., Lu, P., Huang, Z., Guestrin, C., Zou, J., 2025. Optimizing generative ai by backpropagating language model feedback. Nature 639, 609–616.

Zamfirescu-Pereira, J.D., Wong, R.Y., Hartmann, B., Yang, Q., 2023. Why johnny can't prompt: How non-ai experts try (and fail) to design llm prompts, in: Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems.

Zhang, H., Liu, X., Zhang, J., 2023. Extractive summarization via chatgpt for faithful summary generation, in: Findings of the 2023 Conference on Empirical Methods in Natural Language Processing.

Zhang, X., Tian, C., Yang, X., Chen, L., Li, Z., Petzold, L.R., 2025. Alpacare: Instruction-tuned large language models for medical application. arXiv:2310.14558.

Zheng, H.S., Mishra, S., Chen, X., Cheng, H.T., Chi, E.H., Le, Q.V., Zhou, D., 2024. Take a step back: Evoking reasoning via abstraction in large language models, in: The Twelfth International Conference on Learning Representations.

Zhou, Y., Muresanu, A.I., Han, Z., Paster, K., Pitis, S., Chan, H., Ba, J., 2023. Large language models are human-level prompt engineers, in: The Eleventh International Conference on Learning Representations.

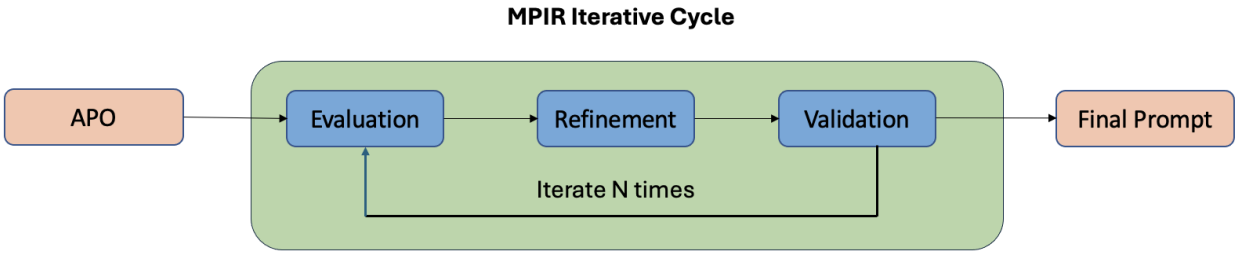

Figure 1: Overview of the Meta-Prompted Instruction Refinement (MPIR) cycle. An APO-generated prompt P0 enters a three-stage loop—Evaluation, Refinement, and Validation—guided by heuristic criteria. Each iteration uses P0 as the base and generates an improved candidate prompt (P1, P2, ..., PN). After N iterations, the prompt with the highest validation score is selected as the final prompt.

```
### Task Overview
You are a Senior Prompt Engineer with two main tasks:
1. Evaluate prompts using a 7-criteria rubric.
2. Refine prompts by applying evaluation feedback.
Always remain objective, precise, and helpful.
### Prompt Under Evaluation
[Prompt]: {prompt}
### Instructions
1. Review Prompt: read the prompt to understand its purpose and structure.
2. Apply Rubric: assess the prompt against the seven criteria below.
3. Score & Justify: for each criterion, assign a score (1-5), state one
strength, one weakness, and a brief rationale.
4. Calculate Total: sum all ratings for a cumulative score out of 35.
5. Recommend Improvements: give 7-10 actionable suggestions.
6. Report Findings: use the standardized output format below.
### Seven-Criteria Evaluation Rubric
1. Role Prompting 2. Step Back 3. Guided Chain of Thought
4. Instruction & Separation 5. Output Format with Examples
6. Worked Reasoning in Examples 7. Conclusion
### Output Format
1. Role Prompting: X/5 - Strength: ... - Improvement: ... - Rationale: ...
(continue for the remaining six criteria)
Total Score: X/35
Refinement Summary (7-10 actionable suggestions): [Suggestion 1] ...
```

Figure 2: Structured meta-prompt for evaluating a candidate prompt: a task overview, evaluation instructions, the seven-criteria rubric, and a standardized output format for consistent scoring and improvement suggestions.

```
Refine the prompt by applying all suggestions from the evaluation.
Make sure to wrap the refined prompt with <START> and <END>
```

Figure 3: Meta-prompt used to refine a candidate prompt by applying all evaluation feedback.

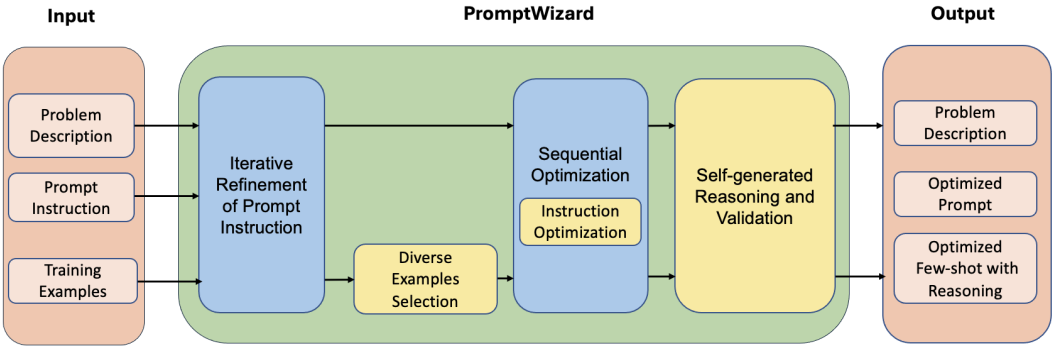

Figure 4: Stage 1: workflow of the PromptWizard base APO, adapted from (Agarwal et al., 2025). It performs iterative refinement of prompt instructions, diverse example selection, sequential optimization, and self-generated reasoning and validation to produce an optimized prompt with few-shot examples and reasoning chains.

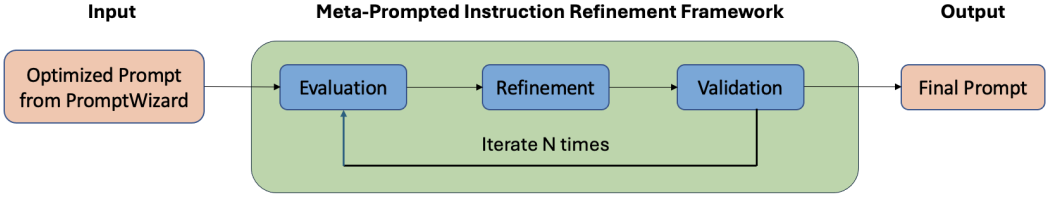

Figure 5: Stage 2: workflow of Meta-Prompted Instruction Refinement (MPIR). MPIR builds on the PromptWizard APO output and performs iterative evaluation, refinement, and validation over N rounds to further improve prompt quality.

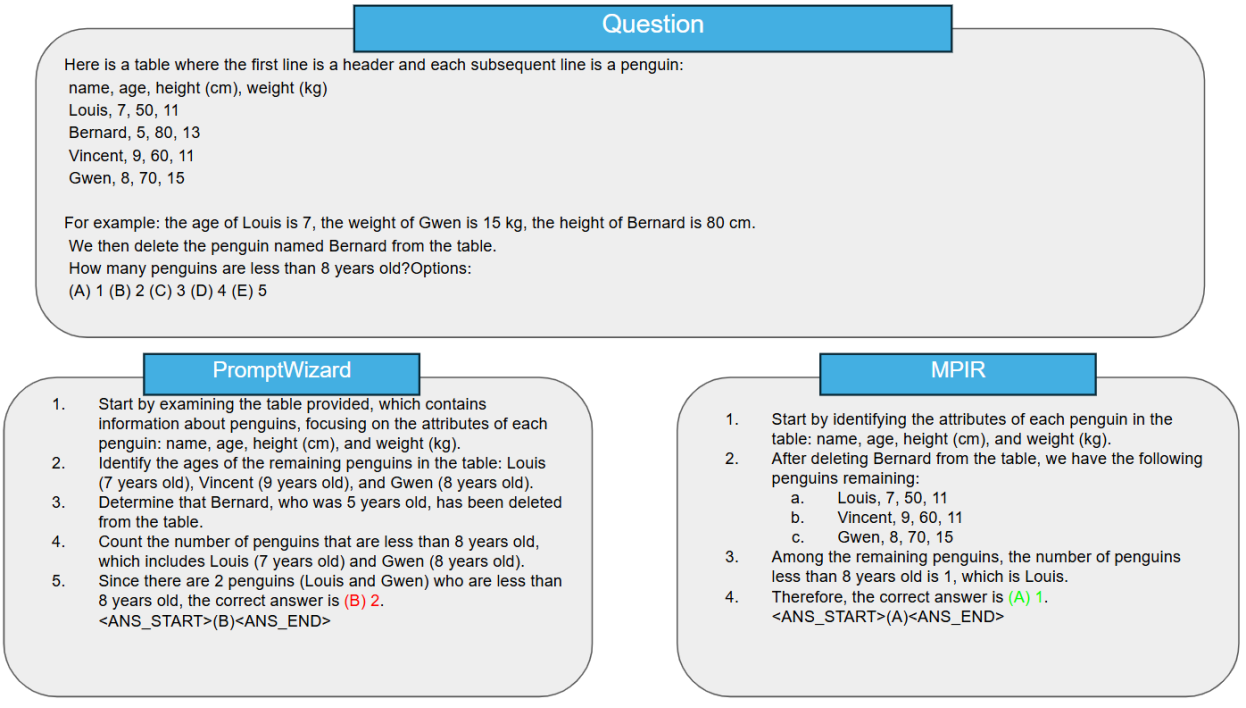

Figure 6: Case study comparing PromptWizard and MPIR on the penguins_in_a_table task. PromptWizard incorrectly identifies two penguins younger than eight years old, while MPIR correctly counts only one.

```
You are given a task that require answering questions about a table
of penguins and their attributes.
Let's think step by step.
For each question, wrap only the final letter (A) (B) (C) (D) (E)
between <ANS_START> and <ANS_END> tags
```

Figure 7: Figure A.1. Zero-shot CoT prompt for penguins_in_a_table.

```
Answer questions about a table of penguins and their attributes.
For each question, present the reasoning followed by final answer
between <ANS_START> and <ANS_END> tags

[Question]: Here is a table where the first line is a header and
each subsequent line is a penguin: name, age, height (cm), weight (kg)
Louis, 7, 50, 11
Bernard, 5, 80, 13
Vincent, 9, 60, 11
Gwen, 8, 70, 15
For example: the age of Louis is 7, the weight of Gwen is 15 kg,
the height of Bernard is 80 cm.
We now add a penguin to the table: James, 12, 90, 12
How many penguins are less than 8 years old?
Options: (A) 1 (B) 2 (C) 3 (D) 4 (E) 5
[Answer]: Let's think step by step.
This question focuses on age. We know the following: Louis is 7
years old, Bernard is 5 years old, Vincent is 9 years old, and Gwen
is 8 years old.
Now, we add James to this table: James is 12 years old.
The penguins that are less than 8 years old are Louis and Bernard.
There are 2 penguins less than 8 years old. So the answer is
<ANS_START>(B)<ANS_END>.

[... two further worked examples omitted for space ...]

For each question, present the reasoning followed by final answer
between <ANS_START> and <ANS_END> tags
```

Figure 8: Figure A.2. Few-shot CoT prompt for penguins_in_a_table (expert-crafted, three worked examples; two omitted here for space, unabridged in the project repository).

```
To ensure accurate comparisons of the penguins' heights, meticulously
analyze and consider the specific attributes of each penguin,
including age, height, and weight. It is crucial to compare the
height of the new penguin, James, with the existing penguins in the
table to arrive at the correct answer. Emphasize the importance of
carefully examining all relevant information about each penguin and
comparing their heights to make an informed decision.

[Question]
Here is a table where the first line is a header and each subsequent
line is a penguin: name, age, height (cm), weight (kg)
Louis, 7, 50, 11
Bernard, 5, 80, 13
Vincent, 9, 60, 11
Gwen, 8, 70, 15
For example: the age of Louis is 7, the weight of Gwen is 15 kg,
the height of Bernard is 80 cm.
Which penguin is younger but taller than Gwen?
Options: (A) Louis (B) Bernard (C) Vincent (D) Gwen (E) James
[Answer]
1. Start by identifying the attributes of each penguin in the table:
name, age, height (cm), and weight (kg).
2. Compare the age and height of each penguin to determine who is
younger but taller than Gwen.
3. Gwen's age is 8, and her height is 70 cm.
4. Louis is younger than Gwen with an age of 7, but his height is
50 cm, so he is not taller than Gwen.
5. Bernard is younger than Gwen with an age of 5, and his height is
80 cm, making him both younger and taller than Gwen.
6. Vincent is older than Gwen with an age of 9, so he is not younger
than Gwen.
7. James is not provided in the table, so he cannot be compared to
Gwen.
8. Therefore, the penguin who is younger but taller than Gwen is
Bernard, making the correct answer (B).
<ANS_START>(B)<ANS_END>

[... two further worked examples omitted for space ...]

For each question, present the reasoning followed by final answer
between <ANS_START> and <ANS_END> tags
```

Figure 9: Figure A.3. PromptWizard-optimized prompt for penguins_in_a_table before MPIR refinement (one of three in-context examples shown; unabridged in the project repository). Note the task-irrelevant aside in the opening paragraph about comparing James's height, discussed in Section 5.2.4.

```
### Context
You are a data analyst tasked with comparing the attributes of
penguins to answer specific questions. Your role is to meticulously
analyze and consider the specific attributes of each penguin,
including age, height, and weight, to ensure accurate comparisons.

### Instructions
1. Understand the Principles: Begin by understanding the principles
of data analysis and comparison techniques. Focus on how to
extract relevant information and compare attributes effectively.
2. Identify Attributes: Examine the table provided, which contains
information about penguins. Identify the key attributes: name,
age, height (cm), and weight (kg).
3. Follow Step-by-Step Reasoning: For each question, follow a
detailed step-by-step reasoning process to arrive at the correct
answer.
4. Use Output Format: Present your reasoning and final answer using
the specified output format, which includes [Question] and
[Answer] sections.
5. Conclude with a Summary: After answering all questions, provide a
concise summary or restatement of the task to ensure you have
completed it correctly.

### Example Output Format
[Question]
Here is a table where the first line is a header and each subsequent
line is a penguin: name, age, height (cm), weight (kg)
Louis, 7, 50, 11
Bernard, 5, 80, 13
Vincent, 9, 60, 11
Gwen, 8, 70, 15
For example: the age of Louis is 7, the weight of Gwen is 15 kg,
the height of Bernard is 80 cm.
Which penguin is younger but taller than Gwen?
Options: (A) Louis (B) Bernard (C) Vincent (D) Gwen (E) James

[Answer]
1. Start by identifying the attributes of each penguin in the table:
name, age, height (cm), and weight (kg).
2. Compare the age and height of each penguin to determine who is
younger but taller than Gwen.
3. Gwen's age is 8, and her height is 70 cm.
4. Louis is younger than Gwen with an age of 7, but his height is
50 cm, so he is not taller than Gwen.
5. Bernard is younger than Gwen with an age of 5, and his height is
80 cm, making him both younger and taller than Gwen.
6. Vincent is older than Gwen with an age of 9, so he is not younger
than Gwen.
7. James is not provided in the table, so he cannot be compared to
Gwen.
8. Therefore, the penguin who is younger but taller than Gwen is
Bernard, making the correct answer (B).
<ANS_START>(B)<ANS_END>

### Conclusion
By following the instructions and using the output format, you can
accurately compare the attributes of penguins to answer the
questions provided. Ensure you have completed the task by reviewing
your answers and the reasoning process.
```

Figure 10: Figure A.4. MPIR-refined prompt for penguins_in_a_table (one of three in-context examples shown, unabridged in the project repository). Compared with Figure A.3, context and instructions are separated into labeled sections and the task-irrelevant aside has been removed.

```
[Question]: If you follow these instructions, do you return to the
starting point? Turn left. Turn around. Turn left. Take 7 steps.
Take 2 steps. Take 4 steps. Take 8 steps.
Options:
- Yes
- No
[Answer]: Let's think step by step.
We start at the origin (0, 0), facing the positive y-axis.
(1) Turn left: (0, 0), facing the negative x-axis.
(2) Turn around: (0, 0), facing the positive x-axis.
(3) Turn left: (0, 0), facing the positive y-axis.
(4) Take 7 steps: (0, 7), facing the positive y-axis.
(5) Take 2 steps: (0, 9), facing the positive y-axis.
(6) Take 4 steps: (0, 13), facing the positive y-axis.
(7) Take 8 steps: (0, 21), facing the positive y-axis.
Since (0, 21) is not (0, 0), we are not where we started.
So the answer is <ANS_START>No<ANS_END>.
```

Figure 11: Figure A.5. Expert-crafted worked example for the navigate task, with explicit running coordinates.

```
[Question] If you follow these instructions, do you return to the
starting point? Take 1 step. Take 7 steps. Take 1 step.
Options:
- Yes
- No
[Answer]
- [Step 1]: Start at the initial point.
- [Step 2]: Take 1 step forward.
- [Step 3]: Move 7 steps forward from the new position.
- [Step 4]: Move 1 step forward from the current position.
- [Logical Pathway]:
- After taking 1 step forward in Step 2, the agent is at a new
position.
- Moving 7 steps forward in Step 3 takes the agent further away
from the starting point.
- Finally, taking 1 step forward in Step 4 continues the movement
away from the starting point.
- [Final Answer]: No <ANS_START>No<ANS_END>
```

Figure 12: Figure A.6. MPIR-refined worked example for the navigate task, tracking state as a bulleted move sequence without explicit coordinates.

```
You are an expert prompt evaluator. Your task is to analyze the
given prompt and provide structured feedback on its overall quality
for guiding a large language model (LLM) to perform the specified
task effectively.
Here is the prompt to evaluate: {prompt}
Assess the prompt based on general aspects of good prompt design,
such as clarity, structure, completeness, and effectiveness in
eliciting the desired LLM behavior. For each aspect you identify
(aim for 5-7 key aspects), provide:
- A score from 1 to 5 (1 = very poor, 5 = excellent).
- A brief justification explaining why you gave that score.
- Strengths: What works well in this aspect.
- Weaknesses: What could be improved in this aspect.
Finally, based on your assessment, suggest 7-10 actionable
improvements to make the prompt better overall. Keep suggestions
practical and focused on enhancing task performance without
overcomplicating the prompt.
Output your response in this exact format:
Aspect 1: [Name of aspect]
Score: [Score]/10
Justification: [Brief explanation]
Strengths: [Bullet points]
Weaknesses: [Bullet points]
[Repeat for each aspect]
Actionable Improvements:
1. [Suggestion 1] 2. [Suggestion 2]...
```

Figure 13: Figure B.1. Meta-prompt for the generic rubric baseline used in Section 5.3.2.

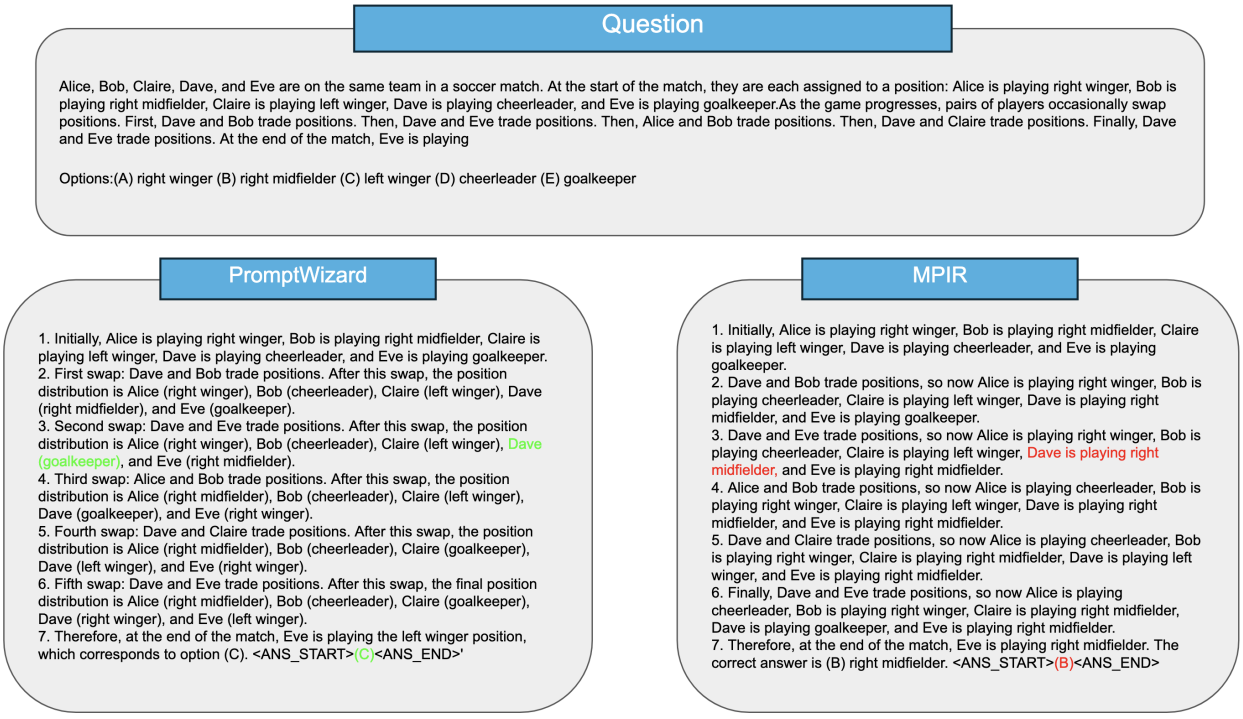

Figure 14: Figure C.1. Case study comparing PromptWizard and MPIR on the tracking_shuffled_objects_five_objects task. Both systems follow the same sequence of swaps but diverge in intermediate reasoning: PromptWizard maintains consistent tracking of player positions and reaches the correct answer (C: left winger), whereas MPIR misapplies one swap and concludes incorrectly (B: right midfielder).

```
You are given a task-solving prompt generated for a Big-Bench Hard
task.

Rewrite the prompt to improve clarity, readability,
and instruction organization while preserving the
original task meaning.

Preserve the original task, answer format, and examples.
Do not change the meaning of the task.
Do not add unrelated content.
Do not use any prompt-evaluation rubric.
Do not mention rubric criteria, scoring, strengths,
weaknesses, or feedback.

Output only the rewritten task-solving prompt.

{variant_instruction}

Original prompt:
<START>
{prompt}
<END>
```

Figure 15: Figure D.1. Meta-prompt for the free rewrite baseline used in Section 4.3.1.